

第十九届全国大学生信息安全竞赛（作品赛）
暨第三届“长城杯”网数智安全大赛（作品赛）

作品报告

☐

命题赛道

☒

自由赛道

作品名称: SC-Sentinel - 基于多智能体的开源软件供应链二进制漏洞审计与验证系统

电子邮箱: 291206750@qq.com

提交日期: 2026 年 7 月 5 日

摘 要

C/C++开源组件广泛应用于操作系统、数据库、网络协议栈、密码学基础设施和嵌入式设备。此类项目通常存在依赖声明分散、构建环境复杂、内存操作风险集中等特点，使供应链风险识别、源码漏洞定位和运行时验证长期处于相互割裂的状态。现有工具多集中于软件物料清单生成、静态告警或模糊测试中的某一环节，难以连续回答“项目引入了哪些组件”“代码中是否存在安全缺陷”以及“缺陷能否在运行时触发”等问题。

针对上述问题，本团队设计并实现 SC-SENTINEL，一套面向 C/C++ 开源软件供应链的多智能体漏洞审计与二进制验证系统。系统由依赖识别智能体、漏洞假设智能体、静态复核智能体、验证工件智能体和报告生成智能体协同完成分析。五类智能体通过结构化中间产物交换信息，将组件识别、漏洞情报关联、函数级语义分析、漏洞假设生成、规则与大语言模型交叉复核、Harness 构建、动态验证和结果归档组织为一条完整的审计链路。

系统以结构化漏洞假设作为规则分析与大语言模型之间的中间表示，通过候选 CWE、可疑代码位置、触发条件和数据流线索约束模型的审计范围；针对不同漏洞类型自动生成包含验证入口 [7, 10]、构建描述、输入种子和适配信息的 Harness 工件；在隔离环境中综合使用 AddressSanitizer、AFL++ 和 eBPF 采集运行时证据，并将动态结果关联到具体静态 Finding。系统进一步对证据来源和证据强度进行区分，使报告能够明确说明每条结论来自静态推断、运行时确认还是多源证据共同印证。

SC-SENTINEL 采用 Vue 3、FastAPI、TaskIQ、Redis、PostgreSQL 和 Docker 构建完整的工程化平台，支持源码压缩包与代码仓库输入、异步任务调度、静态与动态两种审计模式、WebSocket 实时进度和结构化报告导出。系统将确定性规则的稳定性、大语言模型的语义分析能力 [9] 以及动态验证的事实性结合起来，实现了从供应链风险识别到源码漏洞定位、从验证工件生成到运行时证据归因、从审计过程追踪到最终报告交付的完整闭环。

关键词： 软件供应链安全；C/C++；多智能体；漏洞审计；Harness；模糊测试；eBPF；证据裁决

目 录

第一章 作品概述	6
1.1 项目背景及研发意义	6
1.2 需求分析	8
1.3 痛点与竞品分析	9
1.4 作品简介	9
1.5 特色综述	11
1.5.1 版本感知的供应链风险识别	11
1.5.2 假设驱动的多智能体审计	11
1.5.3 面向复现的 Harness 工件自动生成	11
1.5.4 多源运行时证据裁决	11
1.5.5 面向交付的工程化审计平台	11
1.6 应用前景	12
1.6.1 CI/CD 流水线持续审计	12
1.6.2 CTF 竞赛与网络安全教学	12
1.6.3 企业供应链安全合规审计	12
1.7 本章小结	12
第二章 系统总体设计	13
2.1 总体设计	13
2.1.1 设计目标与原则	13
2.1.2 分层架构与职责划分	14
2.1.3 任务生命周期与环境隔离	15
2.2 功能设计	15
2.2.1 审计流水线总体设计	15
2.2.2 依赖分析与供应链风险识别	16
2.2.3 语义切片、漏洞假设与 LLM 审计	17
2.2.4 Harness 构建、动态验证与报告聚合	17
2.3 数据库设计	18
2.3.1 数据实体与关联关系	18
2.3.2 数据写入、完整性与查询策略	19
2.4 接口设计	19
2.4.1 前端与后端服务接口	19
2.4.2 内部通信、Agent 契约与配置管理	20
2.5 本章小结	21
第三章 核心技术实现	22
3.1 创新点一：漏洞假设驱动的五 Agent 级联审计模型	22
3.1.1 漏洞假设中间层设计动机	22
3.1.2 审计追踪链与证据路径构建	23
3.2 创新点二：CWE 感知的自动化 Harness 生成方法	24
3.2.1 基于 CWE 的 Harness 策略选择	24
3.2.2 目标源码与验证入口分离编译机制	25

3.2.3 多级 Seed 生成策略	26
3.2.4 Harness 质量门控机制	27
3.3 创新点三：基于 ASan、AFL++ 与 eBPF 增强融合的动态证据归因与反馈校正机制	27
3.3.1 ASan、AFL++ 与 eBPF 多源动态证据采集机制	28
3.3.2 动态证据强弱分级与状态判定	29
3.3.3 基于 eBPF 反馈的漏洞类型校正机制	31
3.4 本章小结	31
第四章 系统实现	33
4.1 系统环境配置	33
4.1.1 项目目录与服务划分	33
4.1.2 环境变量配置	33
4.1.3 Docker Compose 全栈编排	34
4.2 Web 前端实现	35
4.2.1 前端技术栈与页面路由	35
4.2.2 首页与项目提交页面实现	36
4.2.3 实时监控页面实现	37
4.2.4 历史记录与报告页面实现	38
4.3 后端服务与异步任务实现	39
4.3.1 FastAPI 接口实现	39
4.3.2 TaskIQ 异步流水线实现	40
4.3.3 数据持久化实现	40
4.3.4 进度推送与 PDF 生成实现	40
4.4 五 Agent 审计服务实现	41
4.5 动态验证沙箱实现	41
4.6 本章小结	42
第五章 作品测试与分析	43
5.1 测试环境与测试方案	43
5.1.1 硬件与软件环境	43
5.1.2 测试集组成	44
5.1.3 测试方法与指标	44
5.2 系统功能测试	45
5.2.1 端到端流水线测试	45
5.2.2 API 与 WebSocket 测试	46
5.2.3 PDF 报告导出测试	47
5.3 漏洞审计效果测试	48
5.4 核心创新点效果测试	49
5.4.1 假设驱动链路的输入压缩效果	49
5.4.2 规则与 LLM 协同测试	50
5.4.3 Harness 与动态验证准备度测试	51
5.4.4 多源证据状态分层测试	51

5.5 性能与资源开销测试	52
5.5.1 端到端耗时	52
5.5.2 阶段耗时分布	53
5.5.3 容器资源占用	54
5.6 对比实验	54
5.7 自动化测试与安全性分析	55
5.7.1 后端自动化测试	55
5.7.2 沙箱隔离测试	56
5.7.3 源码生命周期与隐私分析	56
5.8 本章小结	57
第六章 项目管理	58
6.1 企业竞争环境分析	58
6.2 企业竞争能力分析	58
6.3 竞品分析	59
6.4 项目前景分析	59
6.5 项目盈利能力分析	60
第七章 总结	61
参考文献	62

第一章 作品概述

1.1 项目背景及研发意义

随着开源软件在现代软件开发中的使用比例不断提高,开源组件已经成为软件系统的重要组成部分。一个完整的软件产品通常由业务代码、第三方库、构建工具、协议实现和基础运行组件共同构成。开源软件能够显著降低开发成本并缩短产品迭代周期,但同时也使漏洞风险沿依赖关系向下游传播。一个基础组件中的安全缺陷,可能被大量项目直接或间接继承,并在缺少明确依赖记录的情况下长期存在。

软件供应链安全的治理重点已经由单纯检查最终程序,逐步转向识别软件由哪些组件构成、组件版本是否受到已知漏洞影响、项目自身代码是否存在新的安全缺陷,以及安全结论能否被有效复核。软件物料清单、已知漏洞管理和安全开发流程也逐渐成为软件交付的重要内容[1]。在这一背景下,建立覆盖组件、源码和运行时行为的连续审计能力,已经成为软件安全建设中的现实需求。

国务院关于产业链供应链安全的规定（中华人民共和国国务院令 第834号）

发布时间：2026-04-08 来源：中国政府网 字号：[大] [中] [小]

《国务院关于产业链供应链安全的规定》已经2026年3月13日国务院第81次常务会议通过，现予公布，自公布之日起施行。

总理 李强

2026年3月31日

国务院关于产业链供应链安全的规定

第一条 为了防范产业链供应链安全风险，提升产业链供应链韧性和安全水平，维护经济社会稳定和国家安全，根据《中华人民共和国国家安全法》、《中华人民共和国对外关系法》、《中华人民共和国反外国制裁法》、《中华人民共和国对外贸易法》等法律，制定本规定。

第二条 产业链供应链安全工作贯彻总体国家安全观，统筹发展和安全，统筹国内国际，推进高水平对外开放，促进全球产业链供应链稳定畅通。

第三条 国家建立健全产业链供应链安全工作机制，统筹协调产业链供应链安全工作。

国务院外交、发展改革、工业和信息化部、公安、国家安全、法治、财政、自然资源、交通运输、农业农村、商务、金融管理、海关、市场监督管理、网信等部门按照职责分工，负责承担产业链供应链安全工作。国务院有关部门加强产业链供应链安全工作协同配合。

省、自治区、直辖市人民政府在国家统筹协调下，负责与本行政区域有关的产业链供应链安全工作。

图 1-1 国务院关于产业链供应链安全的规定

C/C++软件供应链具有较强的特殊性。操作系统内核、数据库引擎、密码学库、网络协议栈、图像与视频编解码库、嵌入式固件以及工业控制软件等关键系统，大量采用C/C++进行开发。OpenSSL、zlib、libpng、SQLite、FFmpeg和libcurl

等基础组件具有广泛的下游依赖关系，一旦出现高危漏洞，往往会带来大规模的版本排查、修复和重新交付工作。

从语言特性看，C/C++允许开发者直接操作指针、手工管理内存，并提供灵活的类型转换和底层资源控制能力。这些特性在带来较高运行效率的同时，也容易形成缓冲区溢出、释放后使用、二次释放、格式化字符串、整数溢出和空指针解引用等问题。此类漏洞通常与具体的内存布局、输入条件、对象生命周期和控制流有关，仅依靠危险函数或关键词匹配难以形成可靠结论[2]。

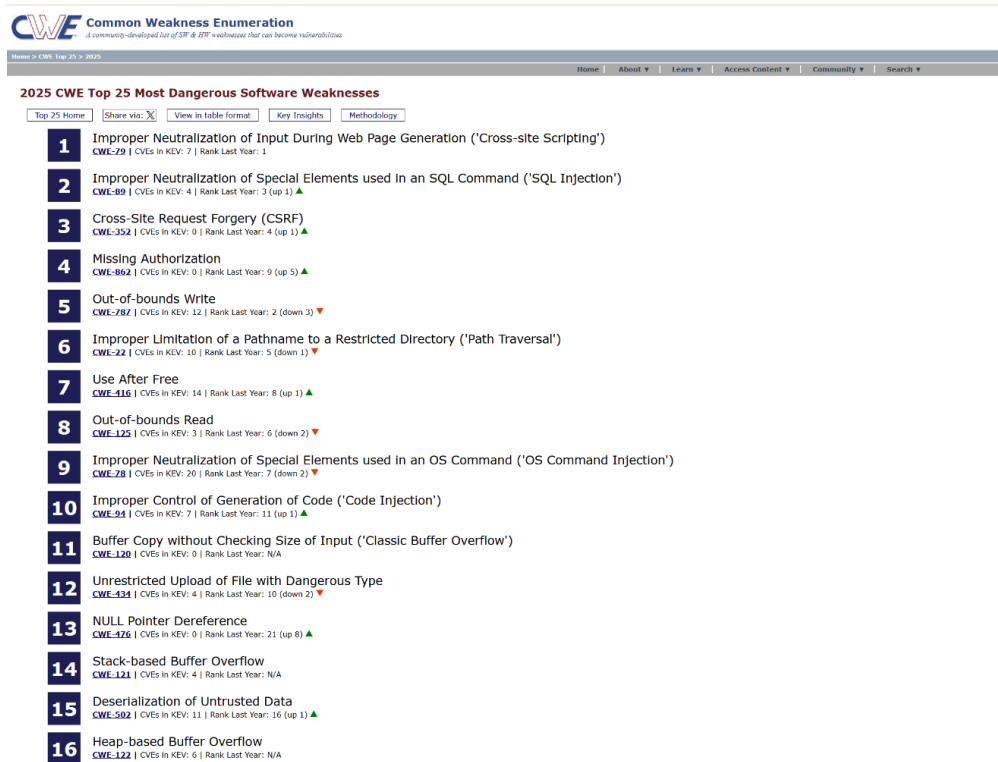


图 1-2 2025 年 CWE top25 最危险漏洞排行榜（资料来源：MITRE[2]）

C/C++项目的依赖识别同样比多数包管理生态更加复杂。项目依赖信息可能同时存在于 CMakeLists.txt、Makefile、vcpkg.json、conanfile.txt、源文件中的 #include 语句和链接参数中，部分第三方代码还会直接存放在项目目录。相同组件可能以头文件路径、构建目标、链接选项或包名称等不同形式出现，导致组件名称归一化、版本判断和漏洞情报匹配存在较大难度。

静态告警与漏洞事实之间也存在明显距离。静态分析可以指出潜在风险位置，却难以直接证明漏洞是否能够在真实路径中触发。C/C++漏洞的复现通常需要安全人员手工编写测试入口、准备输入样本、补充头文件路径、处理目标工程入口冲突并分析运行日志。即使已经获得一条较准确的静态 Finding，后续验证仍然需要较高的工程成本。

近年来，大语言模型在代码理解、漏洞解释和修复建议生成方面展现出较强能力，为传统规则难以处理的复杂语义关系提供了新的分析手段[3]。但直接让大语言模型对整个代码仓库进行开放式审计，容易受到上下文规模、输出稳定性和

模型幻觉影响。SC-SENTINEL 没有将大语言模型作为独立裁判，而是将其纳入规则线索、漏洞假设和动态证据共同约束的审计过程，使模型主要承担语义复核和安全解释工作。

本作品的研发意义主要体现在三个方面。第一，系统面向 C/C++ 项目建立了从依赖风险到源码漏洞的连续分析能力，能够同时处理已知组件风险和项目自身代码问题。第二，系统将静态 Finding 自动转换为标准化验证工件，降低了 Harness 编写和运行环境准备的成本。第三，系统通过结构化中间产物和运行时证据增强 AI 辅助审计的可解释性，使最终结论能够被追踪、复核和再次验证。

1.2 需求分析

当前 C/C++ 开源供应链安全审计缺乏一套能够将依赖识别、静态审计、验证工件生成和运行时证据归因完整串联的系统。结合实际使用场景，SC-SENTINEL 需要满足以下需求。

在依赖识别方面，系统需要从 CMakeLists.txt、Makefile、vcpkg.json、conanfile.txt 和源码 #include 语句等多类载体中提取第三方组件信息，并对同一组件的不同表示方式进行归一化。系统还应识别组件版本，将组件与 OSV、NVD 等公开漏洞情报进行关联，同时保留识别来源和匹配依据[4-5]。

在源码审计方面，系统需要从大量 C/C++ 文件中定位高风险函数，提取函数签名、函数体、调用关系、内存操作和必要的上下文信息。审计不能停留在危险函数匹配层面，还需要分析输入、长度、缓冲区、内存分配和释放等要素之间的关系。

在大语言模型使用方面，系统需要建立明确的输入边界和输出格式。模型应围绕给定的漏洞假设和代码上下文进行判断，并输出漏洞类型、触发条件、证据和修复建议。模型不可用或输出不符合要求时，系统仍应保留确定性的规则分析能力，保证审计流程能够继续完成。

在动态验证方面，系统需要将静态 Finding 转换为可进入运行环境的 Harness 工件。工件应包含测试入口、构建配置、初始种子和目标函数信息，并能够根据漏洞类型选择不同的输入策略。动态执行应在隔离环境中进行，避免待审计源码直接影响业务服务和宿主环境。

在证据管理方面，系统需要将 ASan 诊断、AFL++ 崩溃样本和 eBPF 事件关联到具体 Finding，区分静态候选、运行时确认和待复核状态[6-8]。报告既要说明发现了什么，也要说明判断依据和证据强度。

在用户体验方面，系统需要提供完整的 Web 交互界面，支持源码提交、审计模式选择、任务进度查看、历史任务管理和报告导出。对于耗时较长的动态验证任务，系统应采用异步方式执行，并向用户持续反馈进度。

1.3 痛点与竞品分析

当前市场上已经存在 Coverity、Fortify、Checkmarx、Clang Static Analyzer、CodeQL[9]、Flawfinder 等静态分析工具[1]，也存在 Syft、Trivy、cdxgen 等软件物料清单工具，以及 AFL++[7]、libFuzzer[10]和 OSS-Fuzz[11]等模糊测试方案。这些工具在各自领域具有较成熟的单点能力，但在 C/C++开源供应链审计场景下仍存在以下问题。

第一，依赖识别与漏洞情报之间存在匹配断点。C/C++项目缺少统一的依赖声明方式，相同组件可能具有多种表达形式。例如 OpenSSL 可能以 openssl/ssl.h、OpenSSL::SSL、-lssl 或包管理器中的 openssl 出现。如果工具只能处理其中一种形式，就容易产生遗漏或重复。对于无法取得准确版本的组件，简单的关键词匹配还可能将不适用于当前版本的漏洞错误关联到项目。

第二，静态告警数量与实际审计价值不完全一致。轻量规则扫描执行速度快、部署简单，但难以处理跨语句和跨函数的内存生命周期；编译器级静态分析精度较高，却依赖完整构建环境和编译数据库。在实际项目中，安全人员往往需要面对大量缺少触发条件和上下文说明的告警。

第三，单独使用大语言模型难以保证结果稳定。大语言模型能够识别复杂语义关系，但开放式代码审计可能产生缺少代码依据的结论。模型输出如果只是一段自然语言，也难以直接进入数据库、验证工件和报告流程。对于需要重复执行和正式交付的项目答辩系统，仅依赖模型即时回答无法满足工程要求。

第四，C/C++ Harness 编写门槛较高。目标函数可能被声明为 static，可能依赖复杂初始化过程，也可能与原项目 main 函数发生入口冲突。头文件搜索路径、编译参数和输入种子也需要根据项目情况调整。传统模糊测试工具能够执行已有 Harness，却通常不负责从静态 Finding 自动生成完整验证工件[10-11]。

第五，静态告警与运行时事实之间存在证据断层。静态工具能够指出可疑代码，AFL++能够生成崩溃输入，ASan 能够报告内存错误，eBPF 能够观测运行行为[6-8]，但这些结果通常采用不同格式。缺少统一关联机制时，安全人员仍需手工判断某个崩溃是否对应某条静态 Finding。

SC-SENTINEL 的优势在于将上述能力纳入同一审计任务。系统不只是汇总多种工具的输出，而是使用结构化标识和智能体协作机制建立组件、假设、Finding、Harness 和运行证据之间的关联，使每个阶段的结果都能继续被下游使用。

1.4 作品简介

SC-SENTINEL 是一套面向 C/C++开源软件供应链的多智能体漏洞审计与二进制验证系统。作品名称中的“SC”取自 Supply Chain，“Sentinel”意为哨兵，体现系统在开源组件进入研发、交付和部署环节前进行安全检查的定位。

系统由前端可视化界面、后端异步任务服务、多智能体审计引擎和动态验证沙箱四部分组成。用户可以提交源码压缩包或代码仓库，设置目标漏洞类型并选择是否启用动态验证。系统随后完成组件识别、CVE 风险关联、源码审计、Harness 生成、动态执行、证据归因和报告输出。

SC-SENTINEL 对外抽象为五类智能体，分别为依赖识别智能体、漏洞假设智能体、静态复核智能体、验证工件智能体和报告生成智能体。工程内部的源码扫描、语义切片、种子生成、质量门控和动态日志解析等能力，作为对应智能体内部工具使用，不单独计算为 Agent。

依赖识别智能体负责分析源码、构建文件和依赖清单，完成组件名称归一化、版本判断和公开漏洞关联。漏洞假设智能体负责提取高风险函数和数据流线索，并将其整理为包含候选 CWE、可疑位置和触发条件的漏洞假设。静态复核智能体通过规则与大语言模型交叉审计假设，输出结构化 Finding。验证工件智能体根据 Finding 生成 Harness 包，在隔离环境中组织 ASan、AFL++ 和 eBPF 验证，并完成动态证据归因。报告生成智能体汇总全部结果，形成项目级风险判断和可追踪审计报告。



图 1-3 SC-SENTINEL 端到端漏洞审计流程

系统采用开放的模型服务和漏洞情报接口。大语言模型不可用时，规则回退路径仍可完成静态审计；外部漏洞情报服务不可用时，系统能够使用缓存和已有组件证据继续生成结果。该设计使系统既能发挥外部服务能力，也能够保持核心审计流程的独立性。

1.5 特色综述

1.5.1 版本感知的供应链风险识别

系统综合分析 CMake、Makefile、vcpkg、Conan 和源码头文件引用，从多个来源识别第三方组件。对于同一组件的不同写法，系统通过名称归一化规则合并结果，并保留清单、构建目标、链接参数和头文件路径等原始证据。

系统进一步区分版本明确和版本未知两种情况。版本明确时执行组件名称与版本联合匹配；版本未知时提供候选风险并标记适用性。该方式既能够发现潜在供应链风险，也避免把单纯关键词命中包装成确定漏洞。

1.5.2 假设驱动的多智能体审计

系统不是直接将完整代码仓库交给模型，而是先提取函数级上下文、危险操作、简单调用关系、内存效应和 source-sink 线索，再由漏洞假设智能体生成明确的审计目标。

静态复核智能体只围绕候选 CWE、可疑代码和触发条件进行判断，使模型从开放式“查找漏洞”转变为受约束的“复核漏洞假设”。该机制提高了审计输入的有效性，也便于对模型结论进行规则校验。

1.5.3 面向复现的 Harness 工件自动生成

系统针对静态 Finding 生成包含 AFL++入口、libFuzzer 入口、Makefile、配置文件、输入种子和说明文档的验证包 [7, 10]。验证工件智能体能够根据释放后使用、二次释放、堆溢出、栈溢出和格式化字符串等不同漏洞类型选择相应策略。

每个 Harness 包都记录目标文件、目标函数、输入模型、构建参数和适配情况，使静态 Finding 能够被转换为独立、可移交、可继续调整的安全验证工件。

1.5.4 多源运行时证据裁决

系统综合 ASan 内存错误诊断、AFL++覆盖驱动崩溃样本和 eBPF 运行事件，对静态 Finding 进行确认、补强或校正 [6-8]。不同来源的动态信息被转换为统一证据结构，并根据目标函数、错误类型、调用位置和 Harness 标识关联到具体 Finding。

相比只给出“存在漏洞”或“未发现漏洞”的简单结果，SC-SENTINEL 能够展示静态依据、动态来源和当前证据强度，使最终报告具有更强的说服力。

1.5.5 面向交付的工程化审计平台

系统建立了从任务提交、异步执行、实时进度、结果持久化到 PDF 报告导出的完整平台能力。任务运行过程由后端统一编排，分析结果存入数据库，前端可以展示组件风险、漏洞详情、验证状态和修复建议。

系统还支持静态与动态两种审计模式，既可以面向代码提交提供快速审计，也可以在版本发布前启用完整动态验证，实现不同使用场景下的灵活配置。

1.6 应用前景

1.6.1 CI/CD 流水线持续审计

SC-SENTINEL 的异步任务模式和结构化报告适合接入 CI/CD 流程。在代码提交或合并请求阶段，团队可以运行静态审计，快速识别新增依赖和高风险代码；在主干合并或正式发布阶段，则可以启用动态验证，对重点 Finding 进行深入复核。

系统生成的项目风险、组件 CVE 和漏洞验证状态可以作为流水线安全门禁依据。当项目出现高危组件或动态确认漏洞时，CI 平台可以阻断发布并提示开发人员修复。该方式能够将安全检查从发布后的集中处置提前到研发阶段。

1.6.2 CTF 竞赛与网络安全教学

释放后使用、二次释放、堆栈溢出和格式化字符串等内存安全问题是网络安全教学和 CTF 训练中的重要内容。传统教学通常直接展示漏洞代码和崩溃结果，学员难以理解从风险线索到实际触发之间的完整过程。

SC-SENTINEL 可以展示依赖识别、漏洞假设、静态复核、Harness 生成和运行证据，使学员了解漏洞为何形成、如何设计验证输入以及运行时工具如何确认漏洞。系统还可用于快速检查教学样例和竞赛题目，辅助确认预期漏洞路径是否能够稳定触发。

1.6.3 企业供应链安全合规审计

企业引入新的 C/C++ 组件或发布包含开源代码的软件时，需要掌握组件版本、已知漏洞和项目自身代码风险。SC-SENTINEL 能够对组件风险和源码 Finding 进行统一管理，为组件准入、版本升级、发布前检查和漏洞复核提供审计依据。

系统保留依赖来源、代码位置、触发条件、验证工件和动态日志，便于安全团队进行内部复核，也能够形成较完整的交付材料。对于敏感项目，系统还可接入本地兼容模型和内部漏洞情报服务，适应不同部署环境。

1.7 本章小结

本章介绍了 SC-SENTINEL 的项目背景、实际需求、行业痛点、作品定位和应用前景。系统面向 C/C++ 开源供应链场景，以五智能体协作为核心，将依赖风险、源码审计、验证工件和运行时证据纳入同一审计任务。相比功能相对独立的单点工具，SC-SENTINEL 更重视各分析环节之间的协同，以及最终安全结论的可解释性、可追踪性和可验证性。

第二章 系统总体设计

2.1 总体设计

2.1.1 设计目标与原则

SC-SENTINEL 面向 C/C++ 开源软件供应链漏洞审计与验证场景，目标是在项目依赖、源代码和运行时行为之间建立连续分析关系。系统不将组件查询、静态检查和动态测试视为彼此独立的工具调用，而是将其组织为同一审计任务的不同阶段，最终输出具有完整上下文的项目风险报告。

系统首先强调可解释性。每条漏洞 Finding 除文件路径和代码位置外，还保留候选 CWE、代码上下文、触发条件、判断依据和修复建议。组件风险记录保留组件名称、版本、CVE 编号、风险等级和匹配来源，动态结果保留对应 Harness、错误类型和运行日志。

第二项原则是可追溯性。系统为切片、假设、Finding、Harness 和动态证据分配稳定标识，并通过这些标识建立关联。用户可以从最终报告追溯到静态 Finding，从 Finding 定位到漏洞假设和函数切片，也可以查看其对应的验证工件和运行时证据。

第三项原则是可复现性。系统将模型配置、规则结果、Harness 描述、输入种子和运行日志纳入任务产物。相同输入能够通过规则回退路径获得稳定的基础结果，动态验证工件也能够被重新编译和执行。

第四项原则是人机协同。规则负责提供稳定的风险召回和基本判断，大语言模型负责补充复杂语义分析，动态工具负责提供运行时事实。系统同时输出置信度、构建状态和人工适配提示，使安全人员能够快速识别需要进一步复核的环节。

第五项原则是环境隔离。待审计源码及生成的二进制不直接在业务服务中运行，而是进入受控的 Docker 验证环境。动态任务设置 CPU、内存、进程数和执行时间限制，减少异常程序对系统其他模块的影响。

设计原则	工程落点	主要产物
可解释	保存漏洞上下文、触发条件和判断依据	Finding、修复建议、证据说明
可追溯	使用稳定标识连接各阶段结果	切片、假设、Harness 和证据关联
可复用	固化分析配置、种子和构建描述	JSON 产物、Harness 包、运行日志
人机协同	输出置信度和人工适配提示	质量事件、适配状态
环境隔离	在受限容器中执行未知代码	沙箱结果和资源状态

表 2-1 总体设计原则与工程落点

2.1.2 分层架构与职责划分

系统采用前端展示层、后端服务层、多智能体审计层和动态沙箱验证层构成的四层架构。各层通过明确的数据格式和调用边界协作，将用户交互、任务编排、分析推理和高风险执行环境相互分离。

前端展示层基于 Vue 3 构建，负责项目提交、审计参数配置、任务进度展示和报告查看。前端通过 HTTP 接口提交与查询业务数据，通过 WebSocket 接收实时进度，不直接访问模型服务、数据库或动态沙箱。

后端服务层由 FastAPI、TaskIQ、Redis 和 PostgreSQL 共同构成。FastAPI 负责接口请求和任务管理，TaskIQ 负责执行耗时较长的异步审计任务，Redis 承担任务队列和进度消息传递，PostgreSQL 保存任务、组件风险、漏洞 Finding 和运行事件。

多智能体审计层以独立服务形式运行，负责依赖分析、漏洞假设、静态复核、验证工件和报告生成。五类智能体之间使用结构化对象交换数据，保证每一阶段的输入、输出和质量状态均可独立检查。

动态沙箱验证层负责 Harness 编译、种子回放、AFL++模糊测试、ASan 诊断和 eBPF 事件采集。沙箱接收验证工件，返回编译状态、崩溃样本、诊断日志和运行事件，由验证工件智能体完成后续证据归因。

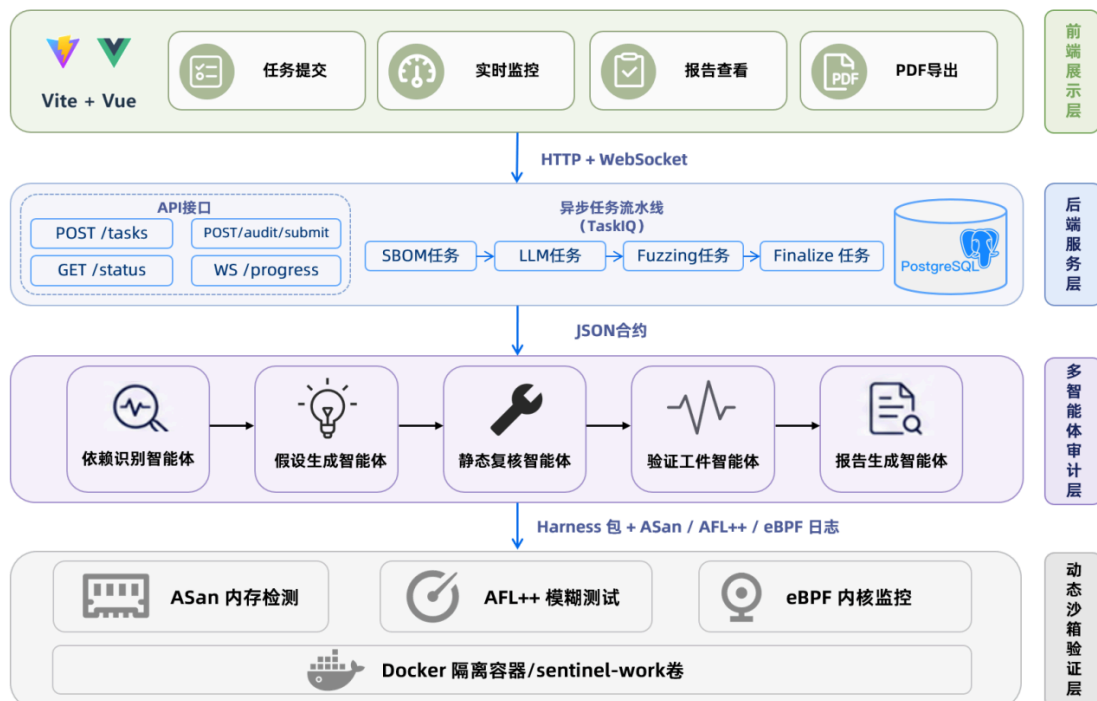


图 2-1 SC-SENTINEL 总体架构

逻辑层	主要职责	协作对象
前端展示层	任务提交、参数配置、进度及报告展示	后端接口、WebSocket
后端服务层	异步编排、状态管理、数据持久化	Redis、PostgreSQL、Agent 服务
多智能体审计层	依赖、假设、复核、工件及报告处理	漏洞情报服务、模型服务
动态沙箱验证层	隔离编译、模糊测试、运行监控	Docker、Harness 工件

表 2-2 逻辑层职责与协作关系

2.1.3 任务生命周期与环境隔离

一次审计任务以项目源码和审计参数为输入。后端创建任务记录后，依次调度依赖识别、源码审计和可选的动态验证。用户未启用动态模式时，任务在静态 Finding 归档后完成；启用动态模式时，验证工件进入沙箱继续执行。

系统对任务状态进行统一管理，使前端能够获知当前阶段、完成进度和异常原因。任务取消或动态验证超时，后端停止继续调度，并尝试终止对应容器和回收临时资源。已经生成的组件风险和静态 Finding 仍然保留，不会因后续阶段失败而全部丢失。

系统按照模块管理配置。后端维护数据库、任务队列、Agent 服务和沙箱参数；多智能体服务维护模型连接和审计策略；动态镜像固化编译器、AFL++、ASan 及相关运行工具。模型密钥和数据库连接信息不会暴露到前端。

对于外部服务，系统设置缓存、超时和降级处理。漏洞情报查询失败时保留组件识别结果；大语言模型不可用时切换到规则审计；动态环境不可用时仍输出静态 Finding 和验证准备信息，从而保证审计任务具有稳定的基础输出能力。

2.2 功能设计

2.2.1 审计流水线总体设计

SC-SENTINEL 采用五智能体协作模型。依赖识别智能体负责建立项目的组件和供应链风险背景；漏洞假设智能体结合函数级语义信息生成候选问题；静态复核智能体通过规则与大语言模型对候选问题进行复核；验证工件智能体生成 Harness 并组织动态验证；报告生成智能体汇总各阶段结果。

智能体	主要输入	核心职责	主要输出
依赖识别智能体	源码、构建文件、依赖清单	组件识别、版本判断、CVE 关联	组件风险清单
漏洞假设智能体	函数切片、调用关系、数据流线索	生成候选 CWE 和触发假设	结构化漏洞假设
静态复核智能体	漏洞假设、目标代码上下文	规则与 LLM 交叉复核	静态 Finding
验证工件智能体	Harness 生成、Finding、目标源码、函数信息	动态执行和证据归因	验证工件、证据状态
报告生成智能体	各阶段结构化结果	风险裁决、追踪链整理	最终审计报告

表 2-3 五智能体职责与产物

五类智能体按照单向数据关系协作，后续智能体使用前一阶段已经形成的结构化结果，不重复解析全部项目。工程内部的源码扫描、函数切片、种子生成、质量门控和日志解析属于智能体内部工具，不单独计算为 Agent。

系统支持按条件执行。模型不可用时，静态复核智能体采用规则结果；用户关闭动态验证时，验证工件仍可作为报告附件生成，但不会进入运行环境；Harness 试编失败时，系统记录构建阻塞信息，不执行无效的模糊测试。

2.2.2 依赖分析与供应链风险识别

依赖识别智能体递归检查项目目录，对 CMakeLists.txt、Makefile、vcpkg.json、conanfile.txt 等文件进行针对性解析，并结合源码头文件引用和链接参数补充组件信息。系统为每条依赖证据标注来源类型。包管理清单和明确构建目标具有较高可信度，头文件引用和链接参数作为补充线索。多种来源共同指向同一组件时，系统合并证据并提高组件识别置信度。

组件名称归一化用于处理不同依赖表示。例如，OpenSSL 相关的头文件、构建目标和链接库被统一到相同组件；常见系统头文件和项目内部相对路径则通过过滤规则排除，减少误识别。

CVE 检索采用版本感知策略。组件版本明确时，系统优先匹配对应版本的公开漏洞记录；组件版本未知时，系统执行候选风险发现并限制返回数量[4-5]。每条风险包含漏洞编号、风险等级、描述和情报来源，供用户进一步核查。系统进一步形成组件风险画像，综合组件置信度、版本置信度、CVE 数量、CVSS 分数和内存安全相关 CWE，确定后续审计优先级。该画像既用于报告展示，也可为漏洞假设阶段提供项目背景。

2.2.3 语义切片、漏洞假设与 LLM 审计

漏洞假设智能体首先调用源码分析工具提取函数签名、函数体、起止行号、相关宏、类型定义和全局变量。对于可识别的函数调用，系统建立简单调用关系，并提取内存分配、释放、返回指针和参数修改等内存效应。

系统根据危险 API、长度运算、数组访问、动态内存管理和格式化输出等特征计算函数风险分值，将函数划分为不同审计优先级。对于高风险函数，系统进一步生成 source-sink 组合和数据流提示。例如，外部输入和长度参数可以构成 source，memcpy、数组写入、释放后访问和非字面量格式串可以构成 sink。

漏洞假设智能体将上述线索整理为结构化假设。针对释放后使用，系统分析对象释放后是否再次读取、写入或传递；针对二次释放，分析同一对象是否在缺少重新分配时再次释放；针对缓冲区溢出，分析缓冲区容量与写入长度是否一致；针对格式化字符串，分析外部可控数据是否被直接用作格式串。

静态复核智能体围绕每条假设调用大语言模型，要求模型基于给定代码判断漏洞是否成立，并给出漏洞类型、触发条件、证据和修复建议。模型结果随后与规则结果进行一致性检查。当模型与规则相互印证时提高置信度；当两者冲突、证据缺失或代码位置异常时，系统记录质量事件并校准结果。

模型未配置、调用超时或无法输出有效结构时，系统直接使用规则结果形成 Finding，保证审计任务不因外部模型服务中断而失去输出。

2.2.4 Harness 构建、动态验证与报告聚合

验证工件智能体根据静态 Finding 生成独立 Harness 包。工件包括 AFL++ 入口、libFuzzer 入口、构建描述、配置文件、种子和说明文档，并保存目标文件、目标函数、候选 CWE 和输入模型等信息。

工件生成后首先进行质量门控。系统检查目标函数原型、入口文件、种子、构建配置和头文件搜索路径，并记录原型匹配置信度、构建准备状态和人工适配要求。质量检查通过的 Harness 可以进入动态沙箱试编。

动态验证阶段使用 ASan 发现内存错误，使用 AFL++ 探索输入空间并保存崩溃样本，使用 eBPF 补充运行行为。验证工件智能体根据 Harness 标识、目标函数、诊断类型和运行上下文，将动态结果关联到具体 Finding。

报告生成智能体汇总组件风险、静态 Finding、Harness 状态和动态证据。报告同时给出项目整体风险和单条 Finding 状态，既方便用户快速判断项目风险，也保留每条结论的具体依据。

2.3 数据库设计

2.3.1 数据实体与关联关系

SC-SENTINEL 以 Task 作为任务管理和结果查询中心。一次 Task 对应一个输入项目、一组审计参数和一组分析结果。组件风险、漏洞 Finding 和 eBPF 事件分别保存，避免将供应链风险、源码问题和原始运行日志混入同一数据结构。

ComponentRisk 用于记录依赖识别智能体发现的第三方组件及其关联 CVE。记录内容包括组件名称、版本、CVE 编号、CVSS 分数、风险等级和识别置信度。Vulnerability 用于保存静态复核智能体形成的 Finding，并承载后续验证状态。主要内容包括文件路径、代码位置、漏洞类型、代码上下文、触发条件、修复建议和动态日志。

EbpEventLog 用于保存验证过程中采集的运行事件，包括事件时间、事件类型、函数名称、内存地址、调用信息和原始数据。完成归因后，事件通过漏洞标识与对应 Finding 建立关系。

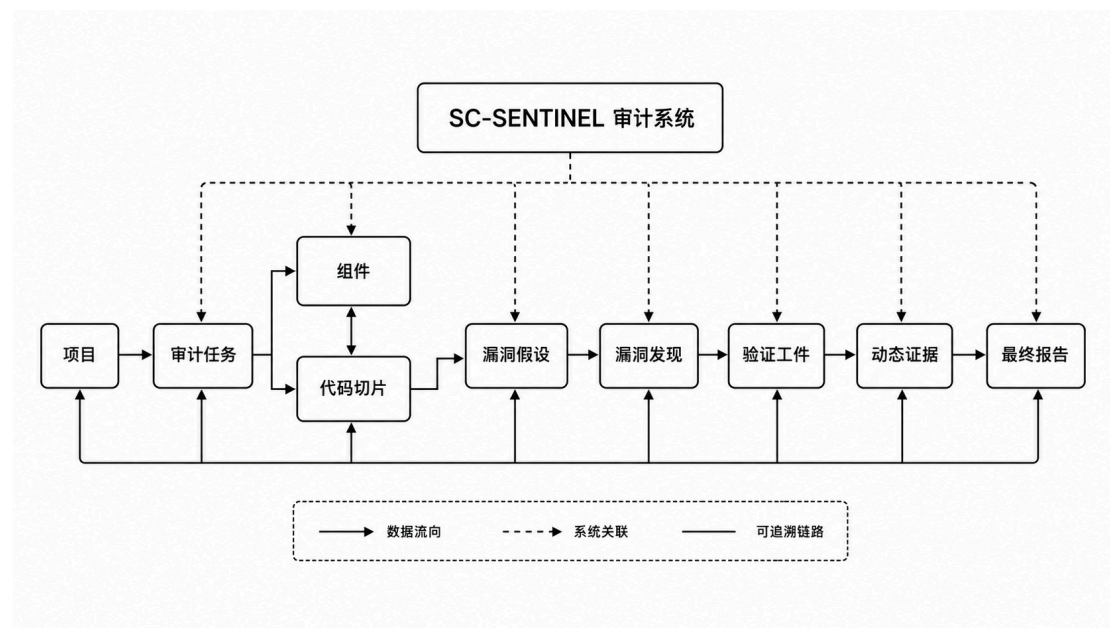


图 2-2 审计数据实体关系

数据实体	主要内容	作用
Task	项目、来源、参数、状态和时间信息	管理一次完整审计
ComponentRisk	组件、版本、CVE、CVSS 和置信度	保存供应链风险
Vulnerability	文件、位置、类型、上下文和验证状态	保存漏洞 Finding
EbpEventLog	事件类型、函数、地址和原始数据	保存运行时行为证据

表 2-4 核心数据实体

2.3.2 数据写入、完整性与查询策略

数据按照审计阶段逐步写入。任务提交时创建 Task 记录；依赖识别完成后写入组件风险；静态复核完成后写入漏洞 Finding；动态验证期间保存运行日志和 eBPF 事件，并更新对应漏洞的验证状态。

Task 与 ComponentRisk、Vulnerability 之间通过任务标识关联，Vulnerability 与 EbpEventLog 之间通过漏洞标识关联。任务列表使用轻量字段和分页查询，报告页面根据任务聚合组件、漏洞和运行事件，保证日常查询效率。

风险等级和验证状态采用统一枚举，避免后端、前端和报告对同一结果产生不同解释。组件风险按照严重程度和 CVSS 排序，漏洞 Finding 按照验证状态、漏洞等级和置信度排序，使高优先级问题优先展示。

2.4 接口设计

2.4.1 前端与后端服务接口

系统接口以任务为中心进行设计。任务创建接口接收项目名称、源码来源、目标漏洞类型和动态验证配置；任务状态接口返回当前阶段、进度和异常信息；报告接口返回组件风险、漏洞 Finding 和动态证据；导出接口生成完整审计报告。

进度通知采用 WebSocket。后端在依赖识别、静态审计、动态验证和任务完成等阶段推送结构化消息，前端实时更新进度和日志。当 WebSocket 连接中断时，前端仍可通过状态接口进行轮询，保证任务状态能够继续展示。

接口类型	主要功能
任务创建	提交源码及审计配置
任务列表	查询历史审计任务
状态查询	获取阶段、进度和错误信息
报告查询	获取组件、Finding 和证据
任务取消	终止未完成任务并回收资源
报告导出	生成完整审计报告
WebSocket 进度	推送实时阶段和日志

表 2-5 面向用户的主要接口

2.4.2 内部通信、Agent 契约与配置管理

后端通过 TaskIQ 和 Redis 调度异步任务，通过 HTTP 调用多智能体服务，通过 Docker 接口管理动态验证容器。前端不直接访问 Agent 服务和 Docker，Agent 服务也不直接修改前端状态。

智能体之间采用结构化 JSON 对象传递结果。组件、切片、假设、Finding、Harness 和证据链接具有稳定字段和唯一标识，使各模块能够独立升级而不破坏整体数据链路。

配置按照模块分别管理。前端只保存服务基础地址；后端管理数据库、任务队列、Agent 地址和沙箱资源；智能体服务管理模型、超时和审计策略；沙箱镜像管理编译器和验证工具。该设计减少了跨模块隐式依赖，也避免敏感配置进入用户交互层。

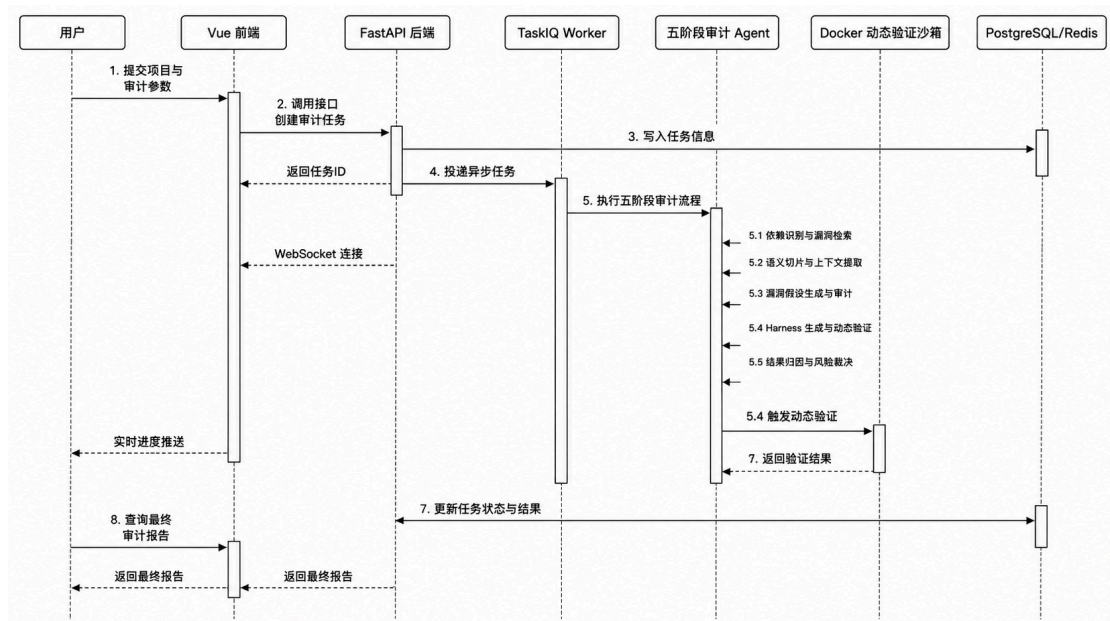


图 2-3 任务提交、执行与结果获取接口时序

2.5 本章小结

本章从设计原则、分层架构、功能模块、数据模型和接口关系五个方面介绍了 SC-SENTINEL 的总体方案。系统采用前端展示、后端编排、多智能体审计和动态验证四层架构，以五类智能体组织组件风险、漏洞假设、静态 Finding、验证工件和运行证据。

系统通过任务化管理解决长时间审计和动态验证的调度问题，通过结构化中间产物保证审计过程可追踪，并通过隔离沙箱控制未知源码的运行风险。各层之间职责清晰、数据关系明确，共同构成 SC-SENTINEL 稳定运行和持续扩展的架构基础。

第三章 核心技术实现

3.1 创新点一：漏洞假设驱动的五 Agent 级联审计模型

SC-SENTINEL 构建了由依赖识别智能体、漏洞假设智能体、静态复核智能体、验证工件智能体和报告生成智能体组成的五 Agent 级联审计模型。五类智能体按照项目风险识别、漏洞推理、静态确认、运行验证和结果裁决的职责分工协同工作。

该模型并不是简单地让多个大语言模型依次生成文本，而是为每个智能体规定明确的输入、输出和质量控制要求。组件信息、函数切片、漏洞假设、静态 Finding、Harness 和动态证据都采用结构化数据表示。每一阶段的结果既是后续智能体的输入，也是最终报告中的可追踪依据。

依赖识别智能体从项目尺度识别供应链风险；漏洞假设智能体将函数切片和风险特征转化为可检查问题；静态复核智能体判断假设是否成立；验证工件智能体将 Finding 转换为可执行验证对象并采集证据；报告生成智能体根据证据强度形成最终结论。

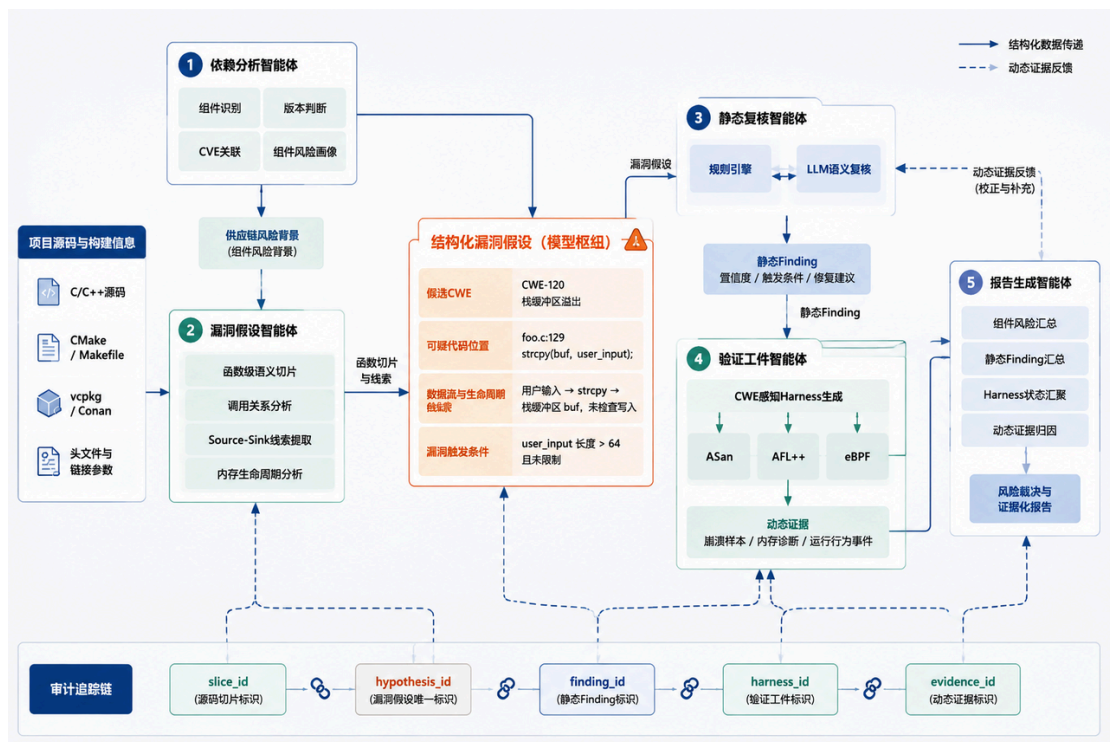


图 3-1 漏洞假设驱动的五 Agent 级联审计模型

3.1.1 漏洞假设中间层设计动机

传统静态扫描通常从危险函数或代码模式直接产生告警。例如，代码中出现 strcpy、memcpy 或 free 并不意味着一定存在漏洞，还需要分析目标缓冲区容量、输入是否可控、对象是否重新分配以及相关路径能否到达。

如果将完整项目直接交给大语言模型，模型需要同时完成风险定位、上下文筛选、漏洞判断和结果解释，容易受到无关代码和上下文规模影响。模型还可能输出缺少代码依据的漏洞类型，难以进入后续验证环节。

SC-SENTINEL 在代码风险线索和静态 Finding 之间引入漏洞假设。漏洞假设不是最终结论，而是对“某段代码可能在何种条件下形成何类漏洞”的结构化表达。每条假设包括来源切片、候选 CWE、可疑代码位置、触发机理、静态证据和初始置信度。

以释放后使用为例，漏洞假设智能体不仅记录代码中出现了 free，还会继续分析同一对象是否在后续语句或被调用函数中再次访问。对于缓冲区溢出，系统同时记录目标缓冲区、写入长度、长度来源和边界检查情况。对于格式化字符串，系统判断格式参数是否为固定字面量，以及外部输入是否能够直接进入格式化函数。

静态复核智能体只接收当前假设和必要代码上下文。大语言模型的任务由开放式“查找漏洞”转变为“复核给定假设”，需要回答漏洞是否成立、触发条件是什么、证据位于何处以及应如何修复。该机制有效缩小了模型推理范围，提高了结果的结构化程度。

3.1.2 审计追踪链与证据路径构建

系统为源码切片、漏洞假设、静态 Finding、Harness 和动态证据分配唯一标识。各对象通过引用字段建立关系，形成完整的审计追踪链。

对象	核心标识	主要记录内容
函数切片	slice_id	文件、函数、行号、代码和上下文
漏洞假设	hypothesis_id	候选 CWE、可疑位置和触发原因
静态 Finding	finding_id	复核结论、置信度和修复建议
验证工件	harness_id	入口、种子、构建参数和质量状态
动态证据	evidence_id	来源、错误类型、关联依据和强度

表 3-1 审计追踪对象

报告中的一条漏洞结论可以反向定位到对应 Finding，再定位到产生该 Finding 的漏洞假设和函数切片。动态证据通过 Finding 和 Harness 标识建立关联，避免将其他验证任务中的崩溃或运行事件错误归入当前漏洞。

依赖识别结果与源码 Finding 保持相对独立。组件 CVE 代表供应链层面的已知风险，源码 Finding 代表项目代码中的安全问题。二者通过同一 Task 和项目上下文统一管理，而不是将存在 CVE 的组件直接解释为当前项目一定能够触发该漏洞。

审计追踪链还保留规则来源、模型来源和回退状态。当模型正常返回时，系统记录模型复核结果；当模型不可用时，系统记录规则回退原因。最终报告能够说明一条 Finding 由何种方式产生，增强审计过程的透明度。

3.2 创新点二：CWE 感知的自动化 Harness 生成方法

静态 Finding 要转化为运行时事实，需要一个能够调用目标代码并接收测试输入的 Harness。传统 Harness 通常由安全人员根据项目结构手工编写，需要处理函数原型、头文件路径、入口冲突、外部依赖和输入种子等问题。

SC-SENTINEL 设计了 CWE 感知的自动化 Harness 生成方法。验证工件智能体综合候选 CWE、漏洞类型、目标函数签名、函数可见性和工程入口结构，选择相应的验证策略，并生成 AFL++ 入口、libFuzzer 入口、Makefile、配置文件、种子和说明文档。

生成的 Harness 不只是一个测试代码文件，而是一套包含目标信息、输入模型、构建方式、种子来源和适配状态的验证工件。该设计使静态 Finding 能够直接进入后续编译和动态验证，也方便安全人员下载后继续修改和复现。

3.2.1 基于 CWE 的 Harness 策略选择

不同漏洞的触发条件存在明显差异，无法使用同一输入模型统一处理。

释放后使用和二次释放通常与对象生命周期及特定控制路径有关。系统需要通过输入字段、调用次数或执行顺序控制对象完成创建、释放和再次访问。

堆缓冲区溢出和栈缓冲区溢出通常与输入长度、目标容量和边界检查有关。系统会重点生成不同长度和边界位置的输入，使 AFL++ 能够快速探索短输入、临界输入和超长输入。

格式化字符串漏洞要求外部输入能够作为格式串本身进入 printf、fprintf、sprintf 等函数。系统在生成 Harness 和种子时保留格式符结构，并加入常见的读取、地址输出和写入类格式符组合。

验证工件智能体还会分析目标函数是否公开声明、是否为 static、参数能否由字节流直接构造，以及项目是否具有可复用的原始入口。目标函数适合直接调用时，系统生成函数级 Harness；目标函数依赖复杂工程上下文时，系统优先使用入口回放或生成适配说明。

漏洞类型	主要控制要素	典型输入策略
释放后使用	对象生命周期、执行路径	控制创建、释放和再次访问
二次释放	释放次数、分支条件	控制同一对象重复进入释放路径
堆/栈溢出	输入长度、缓冲区容量	边界长度和超长数据
格式化字符串	格式串可控性	%x、%p、%s、%n 等组合

表 3-2 典型 CWE 与 Harness 策略

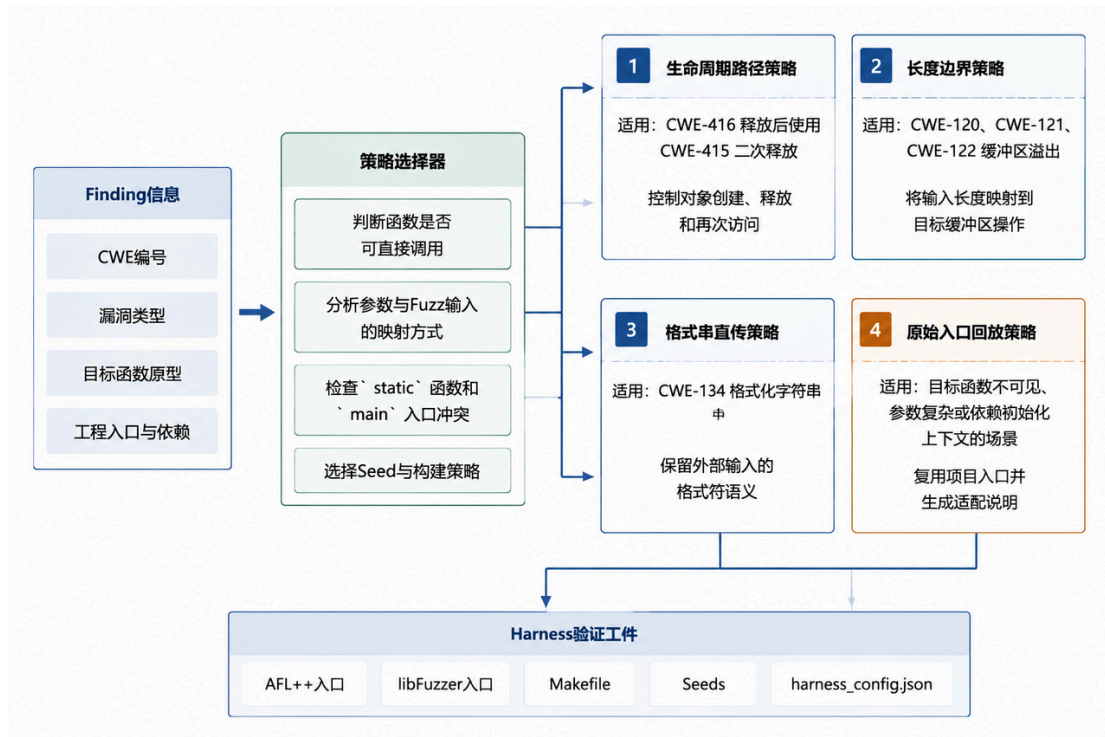


图 3-2 CWE 感知的 Harness 策略选择

3.2.2 目标源码与验证入口分离编译机制

C/C++项目通常包含原始 main 函数，AFL++ Harness 也需要提供测试入口。如果将目标工程和 Harness 直接作为一个编译单元处理，容易产生入口函数重复定义，导致验证程序无法构建。

SC-SENTINEL 采用目标源码与验证入口分离编译机制。目标工程源码和 Harness 分别编译，原项目入口重命名仅作用于目标源码，Harness 入口不受影响，最后再将两部分链接为验证二进制。

该机制避免了对整个编译过程统一替换 main 所造成的入口污染，也保留了目标工程原有函数和全局逻辑。对于存在多个源文件的项目，系统识别包含入口定义的文件，并为不同目标文件生成相应的编译规则。

系统同时扫描项目中的头文件目录和源码目录，生成必要的-I 搜索参数，并保存目标源文件列表。对于具有常规 CMake 或 Makefile 结构的中小型项目，生成的验证工件能够直接进入试编阶段；对于依赖代码生成、专用 SDK 或复杂初始化的项目，工件中会给出明确的适配位置。

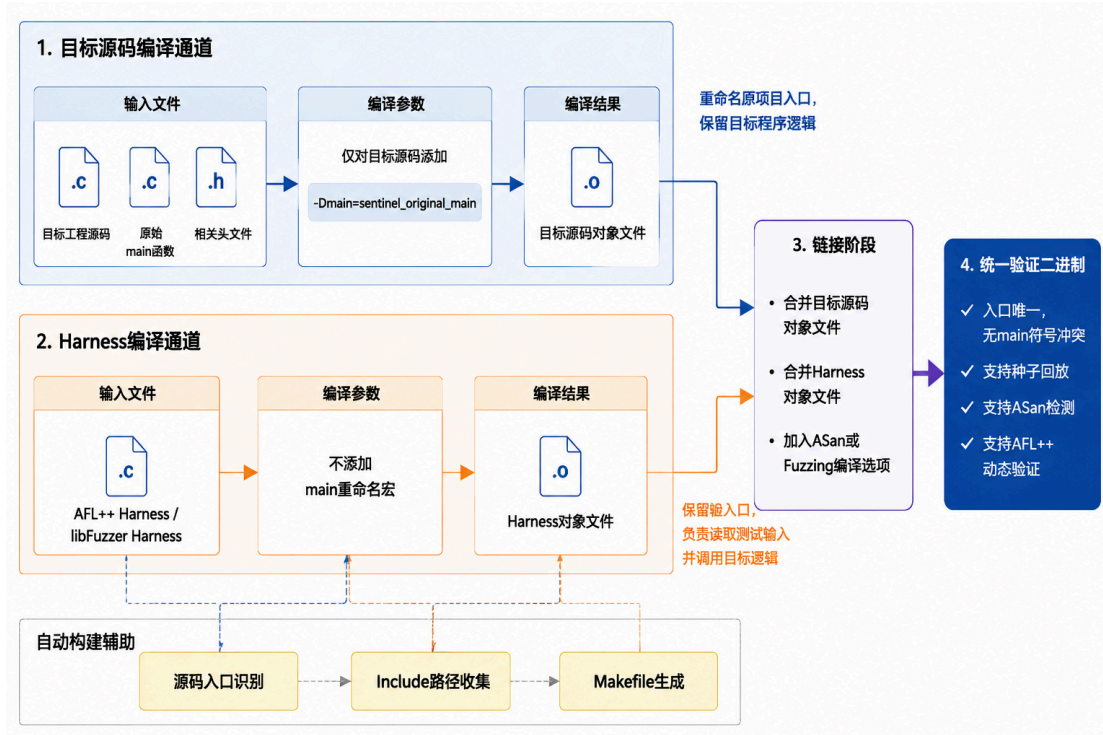


图 3-3 目标源码与 Harness 分离编译机制

3.2.3 多级 Seed 生成策略

初始种子的质量直接影响模糊测试能否快速进入目标代码路径。完全随机的字节序列通常难以通过文件头、长度字段、校验和或协议状态检查，导致有限时间内无法触发深层漏洞。

SC-SENTINEL 按照项目输入信息和漏洞类型组织多级 Seed 生成。系统优先复用项目已有的测试样本和 Corpus，因为这些输入通常能够通过基础解析流程并覆盖真实代码路径。项目中常见的 seeds、samples 和 corpus 目录都可以作为种子来源。

在能够取得目标函数、触发条件和输入模型时，系统根据当前 Finding 生成上下文相关种子。对于结构化输入，种子保留必要的格式字段；对于长度相关漏洞，种子覆盖正常、临界和超长数据；对于格式化字符串，种子包含不同类型的格式符组合。

当项目没有可复用样本，且上下文信息不足时，系统使用 CWE 模板生成基础种子。模板种子保证 AFL++ 具有可启动的初始语料，并为常见漏洞提供具有针对性的边界输入。

系统记录每个种子的来源和用途。报告和 Harness 说明文件能够区分项目原生 Corpus、上下文生成种子和 CWE 模板种子，便于后续复现和人工补充。

3.2.4 Harness 质量门控机制

自动生成 Harness 后，系统需要判断其是否具备进入动态验证的条件。SC-SENTINEL 设计了 Harness 质量门控机制，对函数原型、构建文件、种子、目标源码和依赖信息进行检查。

`prototype_confidence` 用于表示当前输入模型与目标函数签名之间的匹配程度。字符串或字节数组参数通常具有较高匹配度，复杂结构体、回调函数和多级指针参数则需要更多适配。

`build_ready` 用于表示工件是否具备进入试编阶段的基本条件。判断内容包括目标文件是否存在、入口是否生成、构建描述是否完整、种子目录是否可用以及目标函数信息是否充分。

`manual_adaptation_required` 用于标识需要人工调整的场景，包括目标函数不可直接访问、初始化上下文缺失、参数模型复杂或依赖特殊构建环境等情况。

`compile_check` 记录实际试编结果和错误日志。构建失败时，系统保存错误位置和阻塞原因，为安全人员继续调整 Harness 提供依据。

质量门控明确区分工件生成、试编准备、实际编译、成功执行和漏洞触发。通过这一机制，验证工件智能体能够对每个 Harness 的当前状态作出准确描述，也使批量动态验证具备可靠的输入基础。

3.3 创新点三：基于 ASan、AFL++ 与 eBPF 增强融合的动态证据归因与反馈校正机制

静态审计能够识别潜在漏洞，但无法直接证明问题是否能够在运行时触发。为此，SC-SENTINEL 将 ASan、AFL++ 和 eBPF 纳入统一验证过程，并建立 Finding 级动态证据归因机制。

ASan 擅长识别具体内存错误并提供调用栈；AFL++ 擅长通过输入变异探索异常路径；eBPF 能够补充程序运行过程中的行为事件。系统对三类工具的输出进行统一解析，并结合当前 Harness 和 Finding 判断证据是否有效。

动态验证的最终结果不是简单的“崩溃”或“未崩溃”，而是包含动态状态、证据等级、证据来源和关联依据的结构化结果。报告生成智能体据此区分已经运行时确认的问题、需要继续复核的问题以及当前条件下未触发的问题。

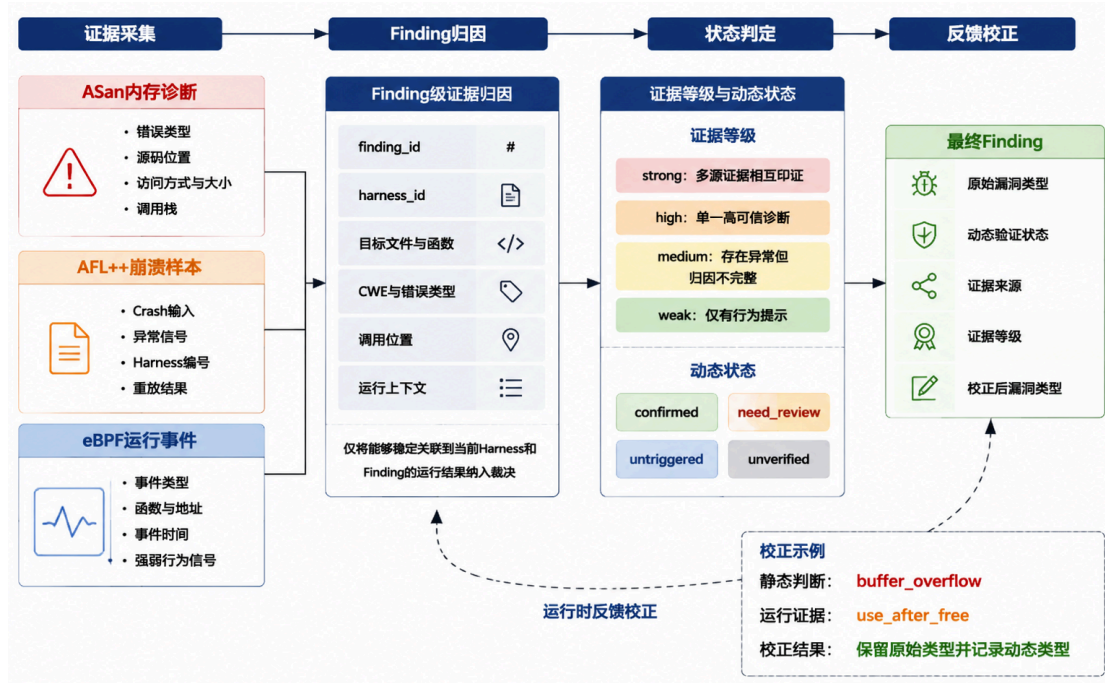


图 3-4 多源动态证据归因与反馈校正机制

3.3.1 ASan、AFL++与 eBPF 多源动态证据采集机制

SC-SENTINEL 在动态验证阶段综合使用 ASan、AFL++和 eBPF，从内存错误诊断、异常输入发现和运行行为观测三个角度采集证据。三类工具承担不同职责：ASan 负责判断发生了何种内存错误，AFL++负责寻找能够触发异常的具体输入，eBPF 负责补充程序执行过程中的关键行为。系统对三类结果进行结构化处理，并统一关联到当前 Harness 和静态 Finding。

(1) 基于 ASan 的内存错误精确识别与证据增强

AddressSanitizer 通过编译插桩检测堆缓冲区溢出、栈缓冲区溢出、释放后使用和二次释放等典型内存错误[6]。验证工件智能体为 Harness 生成对应的 Sanitizer 构建目标，并在种子回放和模糊测试过程中持续收集诊断日志。

系统从 ASan 日志中提取错误类型、访问方式、访问大小、源码位置和调用栈等信息，并将其转换为结构化证据。结构化结果可以与 Finding 中的候选 CWE、目标文件、目标函数和代码位置进行自动匹配，避免安全人员仅依靠原始日志手工定位问题。

当 ASan 错误类型、调用位置和当前 Harness 均与 Finding 一致时，系统将其视为高可信运行时证据，并据此更新 Finding 的验证状态。原始诊断日志同时保留在任务结果中，用于漏洞复现和人工复核。

(2) 基于 AFL++的输入级动态验证

AFL++通过覆盖率反馈持续变异初始种子,探索新的程序执行路径[7]。系统为每个 Harness 建立独立的输入语料和结果目录,记录运行状态、覆盖变化、崩溃样本和相关日志,避免不同 Finding 之间的验证结果相互混淆。

发现崩溃后,系统结合 Harness 标识、目标函数、ASan 诊断和 Finding 类型进行归因。能够稳定重放且与目标 Finding 一致的崩溃样本会作为验证产物进入报告;无法确定根因的异常样本则保留为待复核证据,不直接改变漏洞类型。

AFL++提供的崩溃输入可以被重新加载到同一 Harness 中,使安全人员能够重复执行并观察相同异常。通过保存实际触发输入,系统将静态 Finding 进一步转化为可复现、可移交的漏洞验证结果。

(3) 基于 eBPF 的运行时行为反馈增强

eBPF 用于采集验证程序运行过程中的关键行为信息[8]。系统重点记录内存分配与释放、异常访问、重点函数执行、内存地址和事件时间等内容,为 ASan 和 AFL++结果提供运行上下文。

根据事件的明确程度,eBPF 证据可以分为强行为事件和弱行为事件。能够明确反映二次释放、释放后访问或特定异常内存操作的事件属于强行为事件;疑似边界写入、格式化函数到达和普通内存操作属于弱行为事件。

eBPF 事件只有在与当前 Harness、目标函数和 Finding 建立稳定关联后,才参与漏洞确认或类型校正。弱行为事件主要用于说明目标路径是否到达、指导种子调整并辅助人工分析,不单独作为漏洞确认依据。

三类工具形成互补关系:ASan 提供明确的错误诊断,AFL++提供实际触发输入,eBPF 提供运行行为和上下文信息。验证工件智能体对三类结果进行统一解析和关联,为动态证据分级提供数据基础。

3.3.2 动态证据强弱分级与状态判定

不同工具产生的运行结果在准确性、可复现性和归因能力方面存在差异。SC-SENTINEL 从证据来源、工具可信度、Finding 关联程度和多源一致性四个方面进行综合判断,将动态证据划分为 strong、high、medium 和 weak 四个等级。

证据等级	判定条件	典型情况	对 Finding 的作用
strong	两种及以上独立运行时证据共同指向同一 Finding	ASan 诊断与 AFL++崩溃样本一致； ASan 诊断与强 eBPF 事件一致； AFL++崩溃与强 eBPF 事件相互印证	支持运行时确认，并形成多源交叉证据
high	单一高可信证据能够明确定位错误类型和代码位置	ASan 报告给出具体错误类型、源码位置和调用栈，且与当前 Harness 和 Finding 一致	支持运行时确认
medium	已观察到明显异常，但漏洞类型或归因关系仍不完整	AFL++产生可重复崩溃但缺少完整诊断；单一强 eBPF 事件与 Finding 基本一致	标记为重点复核对象，保留输入和运行事件
weak	仅获得行为提示，或未产生可归因的动态异常	弱 eBPF 事件、普通内存操作、疑似 sink 到达、限定时间内未触发异常	不直接确认漏洞，用于路径分析和验证优化

表 3-3 动态证据等级划分

证据等级描述的是“当前运行结果具有多强的证明能力”，动态状态描述的是“该 Finding 经过验证后处于什么阶段”。二者相互关联，但含义不同。系统使用 confirmed、need_review、untriggered 和 unverified 表示 Finding 的动态状态。

动态状态	判定条件	状态含义
confirmed	获得 strong 或 high 级证据，并能够归因到当前 Finding	漏洞已经获得明确运行时证据支持
need_review	存在 medium 级异常证据，但漏洞类型或关联关系仍不完整	已观察到风险，需要继续复核
untriggered	Harness 已成功执行，但在当前种子和时间预算内未触发可归因异常	本轮验证未复现，不代表静态 Finding 错误
unverified	Harness 尚未完成有效编译或执行，或者验证条件不完整	当前缺少有效动态验证结果

表 3-4 Finding 动态状态判定

系统首先判断动态证据能否关联到当前 Finding，再计算证据等级，最后更新动态状态。该判定顺序避免仅凭崩溃、单条 eBPF 事件或一次未触发结果直接作出漏洞结论。

对于获得 strong 或 high 级证据的 Finding，系统将其标记为 confirmed；对于仅获得 medium 级证据的问题，标记为 need_review；Harness 已经有效执行但未触发异常时，标记为 untriggered；未完成有效编译或运行时，保留为 unverified。

通过证据等级与动态状态的分离，SC-SENTINEL 既能够体现多源证据的可信程度，也能够准确描述每条 Finding 所处的验证阶段，使最终报告更加清晰。

3.3.3 基于 eBPF 反馈的漏洞类型校正机制

大语言模型在分析复杂内存错误时，可能将不同问题概括为缓冲区溢出，但实际运行行为可能属于释放后使用或二次释放。若直接采用模型初始分类，最终报告可能无法准确描述漏洞根因。

SC-SENTINEL 建立了基于运行时反馈的类型校正机制。当强 eBPF 事件或 ASan 诊断与静态分类不一致时，系统首先验证动态证据是否来自当前 Harness，并检查函数、地址和错误类型能否对应当前 Finding。

满足关联条件后，系统保留模型原始类型，同时记录运行时反馈类型、证据来源和校正标志。最终报告展示校正后的主要分类，并在审计追踪信息中保留原始判断。

这一机制没有简单覆盖模型输出，而是完整记录判断变化。安全人员既可以看到静态复核智能体最初如何分类，也可以查看验证工件智能体依据何种运行证据进行了调整。

通过运行时反馈校正，大语言模型在系统中承担语义分析角色，而动态证据承担事实验证角色。两者相互配合，提高了最终漏洞分类和风险结论的可信度。

3.4 本章小结

本章介绍了 SC-SENTINEL 的三项核心技术。

首先，系统构建了漏洞假设驱动的五 Agent 级联审计模型。依赖分析智能体、漏洞假设智能体、静态复核智能体、验证工件智能体和报告生成智能体通过结构化中间产物协同工作，使代码风险线索能够逐步转化为可追踪的漏洞结论。

其次，系统设计了 CWE 感知的自动化 Harness 生成方法。验证工件智能体根据漏洞类型、函数原型和项目结构选择验证策略，并通过分离编译、多级 Seed 和质量门控完成从静态 Finding 到标准化验证工件的转换。

最后，系统建立了 ASan、AFL++ 和 eBPF 协同的动态证据归因与反馈校正机制。多类运行结果被统一关联到具体 Finding，并根据证据强度更新动态状态。运行时事实还可以对静态漏洞类型进行校正，同时保留完整的判断过程。

上述技术共同形成了从供应链风险识别、漏洞假设生成、静态复核、验证工件构建、运行时证据归因到风险报告输出的完整闭环。五类智能体在统一的数据链路上协同工作，使组件风险、源码漏洞和动态验证结果能够相互关联，为 SC-SENTINEL 实现可解释、可追踪、可验证的 C/C++ 开源软件供应链安全审计提供了核心技术支撑。

第四章 系统实现

SC-SENTINEL 采用前后端分离与容器化部署架构，通过模块化服务完成用户交互、异步任务调度、五智能体审计、数据持久化和动态验证。系统由 Vue 3 前端、FastAPI 后端、TaskIQ Worker、五 Agent 审计服务、PostgreSQL、Redis 以及 Docker 动态验证沙箱组成，各模块围绕统一任务标识协同运行。

4.1 系统环境配置

4.1.1 项目目录与服务划分

SC-SENTINEL 按照功能职责划分为前端、后端和智能审计三个核心工程，并在根目录提供全栈编排与项目说明文件。主要目录结构如下：

SC-SENTINEL/

- | - sentinel_frontend # Vue 3 前端交互与报告展示
- | - sentinel_backend # FastAPI、TaskIQ、数据库与沙箱调度
- | - sentinel_agent # 五 Agent 审计引擎、规则与测试样例
- | - docker-compose.yaml # 全栈容器编排入口
- | - INTEGRATION_GUIDE.md # 模块联调说明
- | - README.md # 项目部署与使用说明

sentinel_frontend 负责项目提交、任务监控、历史记录和报告展示；sentinel_backend 负责源码接收、任务创建、异步调度、状态管理、数据持久化和动态容器调用；sentinel_agent 负责依赖分析、漏洞假设、静态复核、验证工件生成、动态证据归因和风险报告聚合。

系统运行产生的上传源码、Harness 工件、动态日志和报告文件统一存放于任务运行目录，不与项目核心代码混合。不同任务使用独立标识和目录，避免中间产物相互覆盖，也方便任务完成后的结果归档和资源清理。

4.1.2 环境变量配置

系统通过环境变量管理大语言模型、数据库、Redis、Agent 服务和动态沙箱等配置。主要环境变量如表 4-1 所示。

配置类别	主要变量	作用
大语言模型	LLM_API_KEY, LLM_BASE_URL, LLM_MODEL	配置兼容模型接口
模型调用	LLM_TIMEOUT, LLM_TEMPERATURE	控制超时和输出稳定性
数据服务	DATABASE_URL, REDIS_URL	配置 PostgreSQL 和 Redis
Agent 服务	ML_AGENT_A_URL, ML_AGENT_B_URL	配置内部审计服务地址
动态验证	SANDBOX_TIMEOUT _SECONDS	控制单次沙箱任务时间
Harness 执行	HARNESSBOX_PACKAGE _TIMEOUT_SECONDS	控制 SPA 单个验证工件的执行时间
联调模式	ML MOCK_MODE	切换真实 Agent 与演示数据

表 4-1 系统主要环境变量

模型密钥通过运行环境注入，不写入代码仓库。模型配置完整时，静态复核智能体调用真实 LLM 完成语义审计；模型未配置、调用超时或结果解析失败时，系统自动保留规则审计结果，保证静态分析任务能够继续完成。

数据库、Redis 和 Agent 地址由 API 服务与 Worker 共同读取。前端只配置后端基础地址，不接触模型密钥、数据库连接和沙箱参数，从配置层面隔离用户交互与内部服务。

4.1.3 Docker Compose 全栈编排

系统使用 docker-compose.yaml 统一编排数据库、消息队列、Agent、API、Worker、前端和动态沙箱镜像[12]。主要服务如表 4-2 所示。

服务	主要职责
PostgreSQL	保存任务、组件风险、漏洞及运行事件
Redis	承担 TaskI/Q 队列、任务结果和进度事件流
Agent	提供五智能体审计能力
API	提供 HTTP 接口、WebSocket 和 PDF 导出
Worker	执行依赖分析、静态审计和动态验证任务
Frontend	提供 Web 页面和反向代理
Sandbox	提供 ASan、AFL++与 eBPF 验证环境

表 4-2 Docker Compose 服务组成

PostgreSQL、Redis、Agent 和 API 均配置健康检查。核心依赖完成初始化后，相关服务才开始处理审计请求，减少多容器并行启动造成的连接失败。Worker 挂载 Docker Socket，根据任务创建短生命周期验证容器；前端通过 Nginx 提供静态资源，并将业务请求转发至后端。

系统支持使用一条 Compose 命令启动主要服务。动态沙箱镜像可以通过独立 Profile 预先构建，避免首次验证任务临时准备工具链。

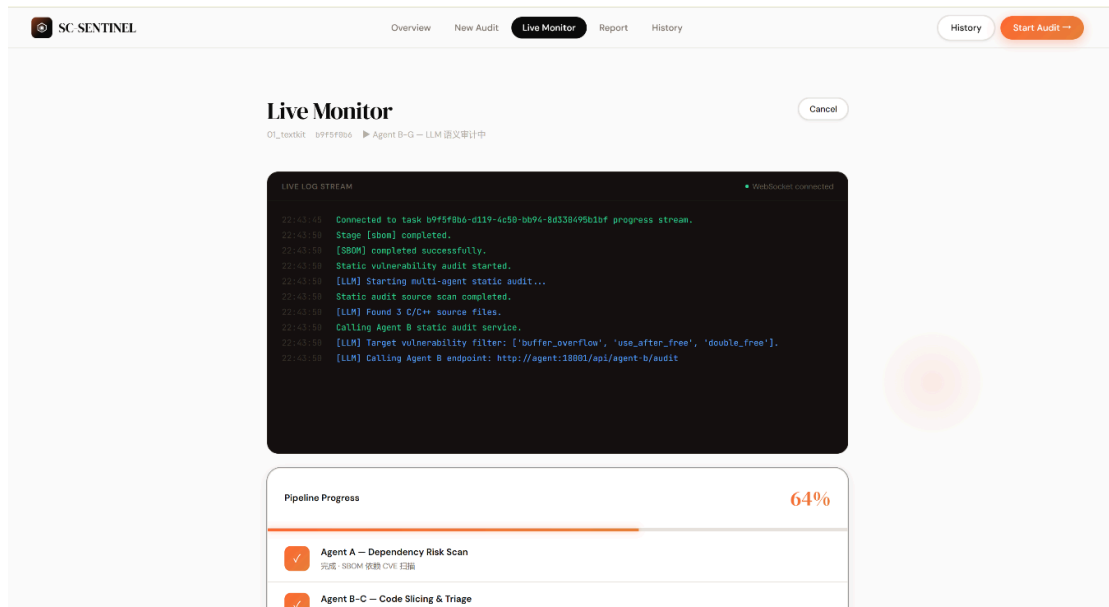


图 4-1 SC-SENTINEL 全栈容器运行状态截图

4.2 Web 前端实现

4.2.1 前端技术栈与页面路由

SC-SENTINEL 前端采用 Vue 3、TypeScript、Vite 和 Pinia 构建，静态资源由 Nginx 提供。系统将首页、项目提交、任务监控、报告和历史记录设计为五个独立页面，页面组件通过路由按需加载。

页面	路由	主要功能
系统首页	/	展示系统能力与任务入口
项目提交	/submit	上传源码并配置审计策略
实时监控	/tasks/:id/monitor	展示任务阶段、进度和日志
审计报告	/tasks/:id/report	展示组件风险、Finding 和动态证据
历史记录	/history	查询并进入历史审计任务

表 4-3 Web 前端页面及功能

Pinia 统一管理当前任务、历史列表、报告数据和连接状态；HTTP 请求通过公共模块封装，统一处理基础地址、响应解析和异常提示。该设计减少了页面之间的重复逻辑，使任务状态和报告数据能够在不同页面间保持一致。

4.2.2 首页与项目提交页面实现

首页围绕五智能体协作、三源动态证据和 Docker 隔离验证展示系统核心能力，并提供“提交项目”和“查看历史任务”两个主要入口。页面同时展示示例组件风险、漏洞状态和动态验证进度，使用户能够快速理解系统的审计对象和结果形式。

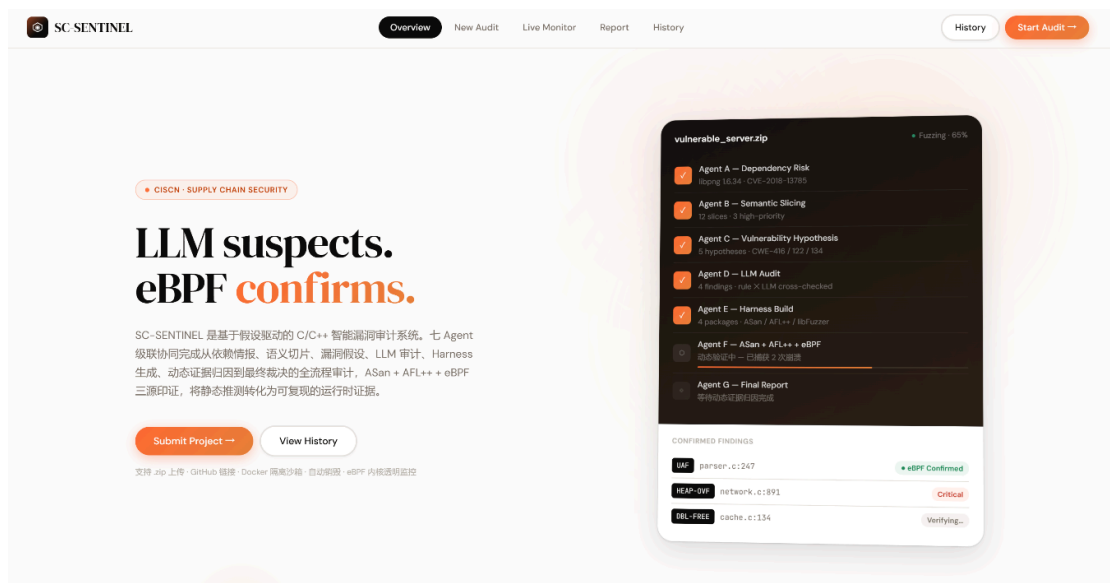


图 4-2 SC-SENTINEL 系统首页

项目提交页面支持 ZIP 源码包和远程代码仓库两种输入方式。ZIP 模式提供拖拽上传和文件选择，前端先检查文件类型和大小；远程仓库模式接收 GitHub、GitLab 等受信任代码托管地址，具体地址安全校验由后端完成。

用户可以选择 Use After Free、Double Free、Heap Buffer Overflow、Stack Buffer Overflow 和 Format String 等重点漏洞类型，并决定是否启用动态验证。未指定目标类型时，系统按照默认规则对支持的漏洞类别进行综合审计。

任务提交分为任务创建和审计启动两个阶段。前端先保存项目基本信息并获取 task_id，随后使用该标识触发异步审计。任务记录与实际执行分离后，系统能够在重复提交、异常恢复、任务取消和历史查询时保持清晰状态。

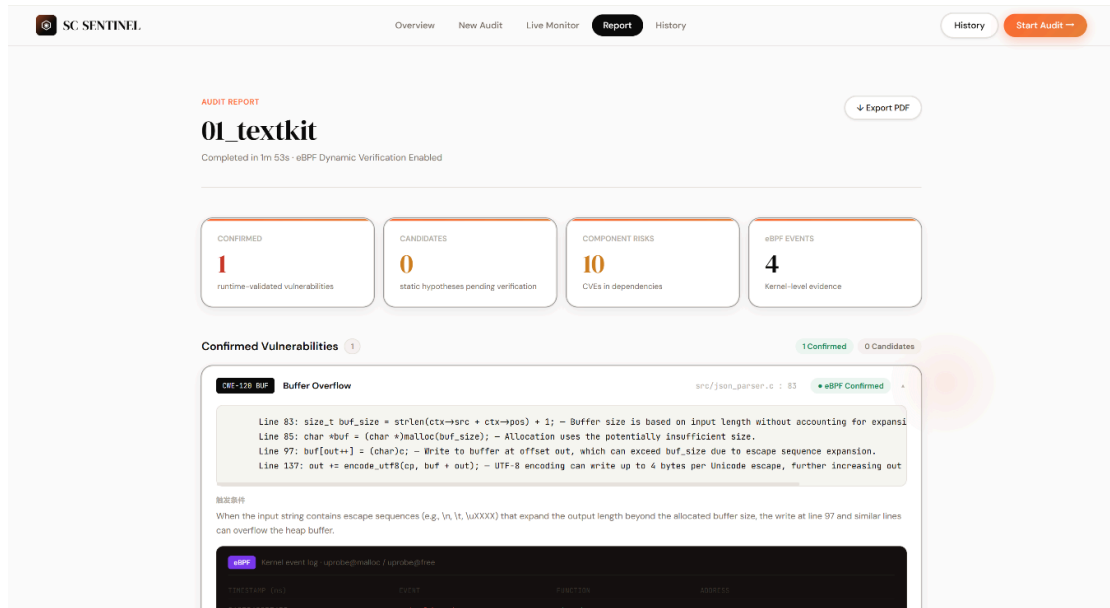


图 4-3 项目提交与审计策略配置页面

4.2.3 实时监控页面实现

实时监控页面根据任务 ID 建立 WebSocket 连接，接收后端推送的阶段、进度、说明和日志。进度消息包含 stage、percent、message 和 log_stream 等字段，前端据此更新阶段卡片、进度条和实时日志窗口。

系统使用 pending、analyzing_deps、llm_auditing、fuzzing、completed 和 failed 表示任务状态。依赖分析阶段负责组件和漏洞情报识别；静态审计阶段完成五智能体分析和 Harness 准备；启用动态模式后，任务进入 fuzzing 状态执行沙箱验证。

前端为 WebSocket 实现心跳与自动重连。当长连接因网络波动或代理配置中断时，页面切换为状态轮询，定期读取任务最新状态。WebSocket 恢复后继续接收增量进度，使长时间动态验证过程能够稳定展示。

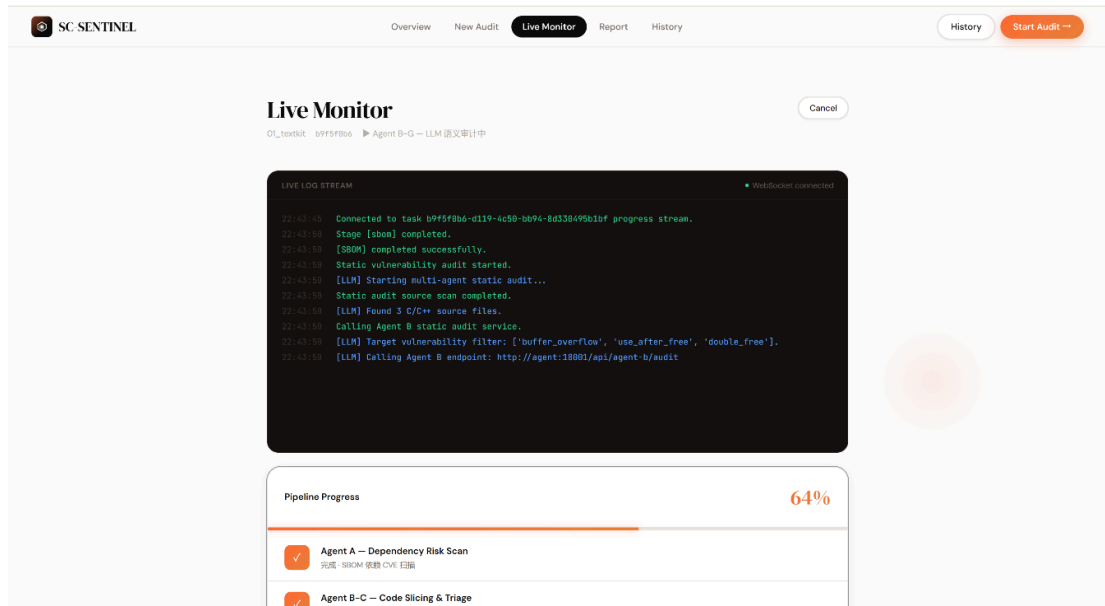


图 4-4 审计任务实时监控页面

4.2.4 历史记录与报告页面实现

历史记录页面展示项目名称、源码来源、动态验证配置、任务状态和创建时间。用户可以从历史记录进入实时监控或报告页面，不需要重新提交项目。

报告页面按照任务概览、组件风险、漏洞 Finding 和动态证据组织内容。组件风险区域展示依赖名称、版本、CVE、CVSS 和风险等级；漏洞区域展示 CWE、文件位置、触发条件、风险等级、验证状态和修复建议；动态证据区域展示 ASan 诊断、AFL++ 日志、eBPF 事件和类型校正信息。

报告中的验证状态采用统一颜色和标签。动态确认问题优先展示，待复核和未触发问题保留静态证据与验证说明。页面同时提供 PDF 导出口，便于项目归档、审计交付和答辩展示。

Component Risk (SBOM) 10 CVEs

Library	Version	CVE	CVSS	Severity	Description
openssl	1.0.1e	CVE-2003-0131	7.5	high	The SSL and TLS components for OpenSSL 0.9.6i and earlier, 0.9.7, and 0.9.7a allow remote attackers to perform an unauthorized RSA private key operation via a modified Bleichenbacher attack that uses a large number of SSL or TLS connections using PKCS #1 v1.5 padding that cause OpenSSL to leak information regarding the relationship between ciphertext and the associated plaintext, aka the "Klima-Pokorny-Rosa attack."
openssl	1.0.1e	CVE-2002-0655	7.5	high	OpenSSL 0.9.6d and earlier, and 0.9.7-beta2 and earlier, does not properly handle ASCII representations of integers on 64 bit platforms, which could allow attackers to cause a denial of service and possibly execute arbitrary code.
openssl	1.0.1e	CVE-2002-0656	7.5	high	Buffer overflows in OpenSSL 0.9.6d and earlier, and 0.9.7-beta2 and earlier, allow remote attackers to execute arbitrary code via (1) a large client master key in SSL2 or (2) a large session ID in SSL3.
openssl	1.0.1e	CVE-2002-0657	7.5	high	Buffer overflow in OpenSSL 0.9.7 before 0.9.7-beta3, with Kerberos enabled, allows attackers to execute arbitrary code via a long master key.
openssl	1.0.1e	CVE-1999-0428	7.5	high	OpenSSL and SSLeay allow remote attackers to reuse SSL sessions and bypass access controls.
openssl	1.0.1e	CVE-2003-0147	5	medium	OpenSSL does not use RSA blinding by default, which allows local and remote attackers to obtain the server's private key by determining factors using timing differences on (1) the number of extra reductions during Montgomery reduction, and (2) the use of different integer multiplication algorithms ("Karatsuba" and normal).
openssl	1.0.1e	CVE-2003-0078	5	medium	ssl3_get_record in s3_pkt.c for OpenSSL before 0.9.7a and 0.9.6 before 0.9.6i does not perform a MAC computation if an incorrect block cipher padding is used, which causes an information leak (timing discrepancy) that may make it easier to launch cryptographic attacks that rely on distinguishing between padding and MAC verification errors, possibly leading to extraction of the original plaintext, aka the "Vaudenay timing attack."

图 4-5 组件风险与漏洞报告页面

4.3 后端服务与异步任务实现

4.3.1 FastAPI 接口实现

后端采用 FastAPI 和异步 SQLAlchemy 实现。API 服务负责源码接收、参数校验、任务创建、审计启动、状态查询、任务取消、报告聚合、PDF 导出和 WebSocket 连接。

接口	方法	功能
/api/v1/tasks	POST	创建任务并接收源码
/api/v1/tasks	GET	查询历史任务
/api/v1/audit/submit	POST	启动异步审计
/api/v1/tasks/{task_id}	GET	查询任务状态
/api/v1/tasks/{task_id}/report	GET	获取结构化报告
/api/v1/tasks/{task_id}/cancel	POST	取消任务并回收资源
/api/v1/tasks/{task_id}/export-pdf	GET	导出 PDF 报告
/api/v1/ws/tasks/{task_id}/progress	WebSocket	推送实时进度

表 4-4 后端核心接口

ZIP 文件采用流式方式保存，避免大文件整体进入内存。远程仓库地址经过协议、域名和本地地址校验，拒绝 file:

4.3.2 TaskIQ 异步流水线实现

依赖查询、模型调用、Harness 生成和 Fuzzing 均属于耗时操作，系统使用 TaskIQ 和 Redis 将其从 HTTP 请求线程中分离。API 接收审计请求后立即返回任务标识和 WebSocket 地址，由 Worker 继续完成后台分析。

静态模式依次完成源码解析、依赖与 SBOM 分析、五智能体静态审计、Harness 落盘、数据持久化和任务归档。动态模式在静态阶段完成后调度沙箱，对生成的 Harness 执行编译、种子回放和模糊测试。

系统通过状态机控制任务推进。静态任务从 pending 进入 analyzing_deps 和 llm_auditing，完成后进入 completed；动态任务在静态阶段后进入 fuzzing。任一阶段出现无法恢复的异常时，Worker 将任务标记为 failed 并保存错误信息。

SBOM 和静态 Finding 采用阶段替换式写入。任务重试时先清理对应阶段的旧结果，再保存本次输出，避免产生重复组件、重复漏洞和失效关联。

4.3.3 数据持久化实现

PostgreSQL 保存系统的长期业务数据，核心模型包括 Task、ComponentRisk、Vulnerability 和 EbpfEventLog。

Task 使用 UUID 作为主键，记录项目名称、源码来源、目标漏洞类型、动态验证配置、任务状态、错误信息和完成时间。ComponentRisk 保存依赖名称、版本、CVE、CVSS 和风险等级。Vulnerability 保存文件位置、漏洞类型、代码上下文、触发条件、修复建议、验证状态和类型校正信息。EbpfEventLog 保存事件时间、事件类型、函数、地址和原始数据。

组件风险和漏洞通过 task_id 归属于同一任务，eBPF 事件通过漏洞标识与 Finding 关联。Redis 主要保存任务队列、短期结果和进度事件，不承担最终报告存储，实现业务数据与高频临时消息分离。

4.3.4 进度推送与 PDF 生成实现

Worker 在阶段切换和动态验证过程中将进度写入 Redis Stream。API 服务读取对应任务的事件流，并转发到浏览器 WebSocket。API 和 Worker 即使运行在不同进程或容器中，也能共享同一任务的实时状态。

每条进度流设置长度上限和有效期，避免长期运行后 Redis 积累过多日志。浏览器建立连接后只订阅当前任务，不同任务之间的进度相互隔离。

PDF 报告由后端使用 ReportLab 生成，内容包括项目元信息、任务耗时、组件风险、漏洞详情、动态验证状态和修复建议。生成器统一处理中文字体、风险颜色、表格宽度和代码片段样式，使导出结果不依赖浏览器打印环境。

4.4 五 Agent 审计服务实现

五 Agent 审计能力运行在独立 FastAPI 服务中。系统保留 `/api/agent-a/analyze` 和 `/api/agent-b/audit` 两个内部接口，以兼容后端现有调用方式；接口内部按照五类业务智能体完成分析。

依赖分析智能体读取 C/C++ 源码、CMake、Makefile、vcpkg 和 Conan 等信息，输出组件、版本、识别证据和 CVE 风险。漏洞假设智能体完成函数级切片、调用关系、内存效应、source-sink 线索和结构化假设生成。静态复核智能体围绕假设执行规则与 LLM 交叉审计，输出 Finding 和质量控制信息。

验证工件智能体根据 Finding 生成 AFL++ 与 libFuzzer 入口、Makefile、种子和配置文件，并将 ASan、AFL++ 和 eBPF 结果关联到具体 Finding。报告生成智能体汇总组件风险、漏洞假设、静态 Finding、Harness 状态和动态证据，形成最终风险结论。

智能体	主要实现能力	主要输出
依赖分析智能体	多来源依赖识别、名称归一化、OSV/NVD 查询	组件风险与风险画像
漏洞假设智能体	函数切片、调用关系、内存效应、假设生成	结构化漏洞假设
静态复核智能体	规则与 LLM 复核、置信度校准	静态 Finding
验证工件智能体	Harness、Seed、质量门控和证据归因	验证工件与动态状态
报告生成智能体	风险汇总、证据链整理和报告生成	最终风险报告

表 4-5 五 Agent 工程实现及主要输出

Harness 文件在 Agent 响应中以可传输形式返回，再由 Worker 写入后端管理的任务目录。这样，后端不依赖 Agent 容器内部路径，动态沙箱可以直接使用持久化后的验证工件。

系统通过切片、假设、Finding 和 Harness 标识保持结果关联。报告中的漏洞能够追溯到对应函数、漏洞假设、静态复核结果和运行时证据。

4.5 动态验证沙箱实现

动态验证由后端 SandboxManager 统一调度。Worker 根据任务 ID 创建短生命周期容器，将源码和 Harness 复制或挂载到受控目录，依次执行编译、种子回放、ASan 检测、AFL++ 模糊测试和 eBPF 事件采集。

系统优先执行 Harness 自带的构建规则。编译成功后先回放生成种子，快速检测能够直接触发的内存错误，再按照任务预算启动 AFL++。验证结束后，系统解析崩溃目录、Sanitizer 日志和 eBPF 事件，并将结果返回 Worker。

默认沙箱采用非特权用户、关闭网络、移除 Linux capabilities、只读根文件系统、no-new-privileges、PID 限制、CPU 限制、内存限制和墙钟超时等策略[12]。任务结束后自动删除容器，减少未知程序持续驻留的风险。

源码摄取阶段同样实施安全检查。ZIP 解压前验证路径穿越、符号链接、文件数量、展开体积和异常压缩率；远程仓库只允许受信任托管平台，并关闭交互式凭据提示。源码不会在 API 或数据库服务中直接编译运行。

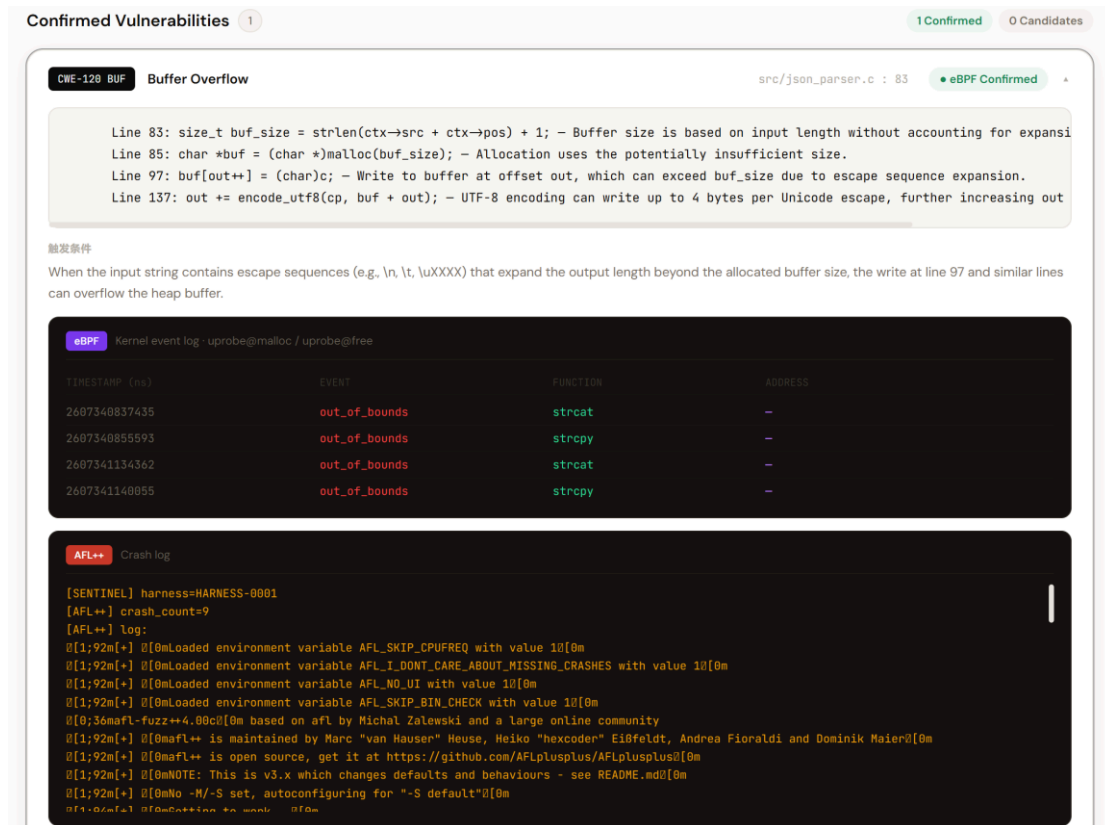


图 4-6 Docker 动态验证沙箱及日志回传

4.6 本章小结

本章介绍了 SC-Sentinel 的工程实现。系统通过 Docker Compose 组织前端、API、Worker、Agent、PostgreSQL、Redis 和动态沙箱，形成统一部署环境；通过 FastAPI、TaskIQ 和 Redis Stream 实现长任务异步调度与实时进度；通过 PostgreSQL 和 PDF 导出完成审计结果的持久化和交付。

五智能体审计服务完成依赖分析、漏洞假设、静态复核、验证工件和报告生成，动态沙箱则负责 Harness 编译、ASan 检测、AFL++测试和 eBPF 事件采集。各模块围绕统一任务标识协同运行，实现了从源码提交到风险报告输出的完整工程链路。

第五章 作品测试与分析

SC-SENTINEL 测试覆盖端到端功能、接口通信、审计效果、核心技术、动态验证、性能开销和系统安全性。测试数据主要来源于项目内置的 Level 1 基础样例、Level 2 工程测试集及其 oracle。

5.1 测试环境与测试方案

5.1.1 硬件与软件环境

测试硬件与软件环境如表 5-1 和表 5-2 所示。

配置项	配置信息
CPU	Intel Core i9-14900HX
内存	32 GB
硬盘	1 TB
GPU	NVIDIA GeForce RTX 4070 Laptop GPU

表 5-1 测试硬件环境

类别	版本或配置
操作系统	Microsoft Windows NT 10.0.26200.0, 64 位
Docker	29.6.1
Docker Compose	5.1.4
Node.js	24.15.0
npm	11.12.1
数据库	PostgreSQL 15-alpine
消息队列	Redis 7-alpine
后端框架	FastAPI、Uvicorn、TaskIQ
前端框架	Vue 3、Vite、Pinia
动态验证镜像	sentinel-sandbox:latest, 约 2.1 GB
LLM 模型	deepseek-v4-flash
LLM 接口	阿里云 DashScope OpenAI-compatible API

表 5-2 测试软件环境

测试环境以 Docker 作为主要运行基础，前端、后端、数据库、队列和 Agent 服务采用容器化部署。外部大语言模型通过兼容接口调用，因此系统静态审计不要求本地 GPU；CPU 和内存资源主要用于源码分析、Harness 编译和动态沙箱执行。

5.1.2 测试集组成

项目内置 20 个 Level 1 单文件 C 语言样例和 14 个 Level 2 小型工程。Level 1 覆盖 Use After Free、Double Free、Heap Buffer Overflow、Stack Buffer Overflow 四类基础漏洞，每类包含不同代码形态，用于验证五智能体主链路和规则稳定性。

Level 2 测试集包含头文件、构建文件、多函数调用和第三方依赖信息，覆盖 CWE-121、CWE-122、CWE-415 和 CWE-416 等类型，主要用于审计效果和动态验证测试。

编号	工程	预期漏洞类型
1	textkit	Heap Buffer Overflow (CWE-122)
2	imagepipe	Use After Free (CWE-416)
3	fieldbus	Stack Buffer Overflow (CWE-121)
4	astcore	Double Free (CWE-415)
5	chunkstream	Integer Overflow 引发 Heap Overflow (CWE-190/ CWE-122)
6	audiodec	Out-of-Bounds Read (CWE-125)
7	imagecodec	Heap Buffer Overflow (CWE-122)
8	proxyroute	Heap Buffer Overflow (CWE-122)
9	resolver	Stack Buffer Overflow (CWE-121)
10	authframe	Stack Buffer Overflow (CWE-121)
11	tunnelctl	Use After Free (CWE-416)
12	sshscan	Use After Free (CWE-416)
13	cachemgr	Double Free (CWE-415)
14	logutil	Format String (CWE-134)

表 5-3 Level 2 核心测试集

level2_oracle 保存预期漏洞、CWE、目标函数、触发条件和部分验证输入，用于核对系统 Finding。vulnerable_project 用于全栈任务提交、报告展示和 PDF 导出演示，不纳入主要漏洞命中率统计。

5.1.3 测试方法与指标

测试首先验证五智能体命令行与 HTTP 服务，再验证后端 API、WebSocket、数据库和前端页面，最后启用 Docker 沙箱执行 Harness、ASan、AFL++ 和 eBPF 验证。

每个 Level 2 工程设置一个主要 oracle 漏洞。当系统输出的漏洞类型、目标文件、目标函数和根因与 oracle 一致时，记为正确命中。14 个工程中共有 9 个主要漏洞被系统识别，主要漏洞命中率为：

每个 Level 2 工程设置一个主要 oracle 漏洞。当系统输出的漏洞类型、目标文件、目标函数和根因与 oracle 一致时，记为正确命中。14 个工程中共有 9 个主要漏洞被系统识别，主要漏洞命中率为：

$$\text{HitRate} = \left(\frac{9}{14} \right) \times 100\% = 64.29\%$$

Precision 和 Recall 的通用计算方式分别为：

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

本次测试采用主要 oracle 命中率作为核心指标。该指标衡量系统能否识别每个工程预设的主要漏洞，不与 Precision、Recall 或 F1 混用。

5.2 系统功能测试

5.2.1 端到端流水线测试

端到端测试从前端提交项目开始，覆盖任务创建、依赖分析、五智能体审计、可选动态验证、状态推送、报告查询和 PDF 导出。测试选择 10 个代表性 Level 2 项目，分别执行静态和动态模式。

样例	模式	状态	耗时	组件风险数	漏洞数	PDF
01_textkit	静态审计	completed	53 s	10	1	通过
01_textkit	动态验证	completed	59 s	10	1	通过
03_fieldbus	静态审计	completed	23 s	10	1	通过
03_fieldbus	动态验证	completed	55 s	10	1	通过
04_astcore	静态审计	completed	26 s	0	0	通过
04_astcore	动态验证	completed	44 s	0	0	通过
05_chunkstream	静态审计	completed	102 s	20	2	通过
05_chunkstream	动态验证	completed	124 s	20	2	通过
07_imagecodec	静态审计	completed	49 s	0	1	通过
07_imagecodec	动态验证	completed	58 s	0	1	通过
08_proxyroute	静态审计	completed	41 s	8	2	通过
08_proxyroute	动态验证	completed	121 s	8	2	通过
09_resolver	静态审计	completed	85 s	0	2	通过
09_resolver	动态验证	completed	130 s	0	2	通过
10_authframe	静态审计	completed	27 s	0	0	通过
10_authframe	动态验证	completed	57 s	0	0	通过
13_cachemgr	静态审计	completed	64 s	10	3	通过
13_cachemgr	动态验证	completed	131 s	10	3	通过
12_sshscan	静态审计	completed	23 s	5	1	通过
12_sshscan	动态验证	completed	43 s	5	1	通过

表 5-4 端到端流水线实测结果

20次任务均进入completed状态,静态模式和动态模式均可生成结构化报告。测试结果表明,源码摄取、异步任务、Agent调用、结果落库、沙箱调度和前端查询能够稳定协同。

5.2.2 API 与 WebSocket 测试

测试覆盖任务创建、审计提交、状态查询、历史列表、报告查询、任务取消、PDF导出和WebSocket进度接口,同时检查参数错误、任务不存在和重复提交等异常情况。

接口或功能	测试结果
GET /health	HTTP 200, status=ok
POST /api/v1/tasks (ZIP)	HTTP 200
POST /api/v1/tasks (远程仓库)	HTTP 200
GET /api/v1/tasks	HTTP 200
GET /api/v1/tasks/{id}	HTTP 200
POST /api/v1/audit/submit	成功下发异步流水线
WS /api/v1/ws/tasks/{id}/progress	连接成功
WebSocket ping/pong	通过
未完成任务请求报告	返回业务错误, 符合预期

表 5-5 API 与 WebSocket 测试结果

测试结果表明, 后端能够正确返回任务状态和业务错误, Worker 进度可经 Redis Stream 转发到前端。WebSocket 断开后, 前端能够使用状态接口继续同步任务, 连接恢复后重新进入实时模式。

5.2.3 PDF 报告导出测试

PDF 测试检查中文字体、项目概览、组件风险、漏洞详情、动态状态、表格分页和文件下载。不同风险等级使用统一颜色, 较长代码上下文能够自动换行, 生成文件可在常用 PDF 阅读器中正常打开。

测试项	结果
HTTP 状态码	200
Content-Type	application/pdf
文件头	%PDF
文件可正常打开	是
completed 任务导出	通过
未完成任务导出保护	通过

表 5-6 PDF 导出测试结果

SENTINEL

C/C++ 开源供应链 eBPF-LLM 协同漏洞审计报告

项目名称	01_textkit
任务 ID	b9f5f0b6-d119-4c50-bb94-8d330495b1bf
创建时间	2026-07-04 14:43:45 UTC
完成时间	2026-07-04 14:45:38 UTC
审计耗时	113.3 秒
动态验证	已开启 eBPF 验证
发现漏洞数	1
组件风险数	10

一、第三方组件风险清单

组件名称	版本	CVE 编号	CVSS	危险等级
openssl	1.0.1e	CVE-2003-0131	7.5	HIGH
openssl	1.0.1e	CVE-2002-0655	7.5	HIGH
openssl	1.0.1e	CVE-2002-0656	7.5	HIGH
openssl	1.0.1e	CVE-2002-0657	7.5	HIGH
openssl	1.0.1e	CVE-1999-0428	7.5	HIGH
openssl	1.0.1e	CVE-2003-0147	5.0	MEDIUM
openssl	1.0.1e	CVE-2003-0078	5.0	MEDIUM
openssl	1.0.1e	CVE-2002-0659	5.0	MEDIUM
openssl	1.0.1e	CVE-2001-1141	5.0	MEDIUM
openssl	1.0.1e	CVE-2000-0535	5.0	MEDIUM

二、漏洞详情清单

#1 [buffer_overflow]	src/json_parser.c:83	验证状态: 已确认
Line 83: size_t buf_size = strlen(cix->src + cix->pos) + 1; --- Buffer size is based on input length without accounting for expansion from escape sequences. Line 85: char *buf = (char *) malloc(buf_size); --- Allocation uses the potentially insufficient size. Line 97: buf[out++] = (char)c; --- Write to buffer at offset out, which can exceed buf_size due to escape sequence expansion. Line 137: out += encode_utf8(cp, buf + out); --- UTF-8 encoding can write up to 4 bytes per Unicode escape, further increasing out beyond buf_size.		

图 5-2 系统生成的 PDF 审计报告

5.3 漏洞审计效果测试

本节以 14 个 Level 2 工程及其 oracle 为统一统计口径。系统输出经过根因级去重后，与预设漏洞类型、目标函数和触发机理进行核对。

指标	数值
oracle 标注漏洞数	14
系统输出去重后主要结论数	14
正确命中数 TP	9
未正确命中数	5
Precision	64.29%
Recall	64.29%
F1	64.29%
可进入 Harness 或动态验证的正确漏洞	6
动态可验证率	66.67%

表 5-7 Level 2 漏洞审计结果

系统正确识别 9 个工程的主要漏洞，主要漏洞命中率为 64.29%。缓冲区溢出、格式化字符串和直接生命周期错误具有较明确的危险操作及数据关系，能够被规则和 LLM 协同链路稳定捕获。复杂整数运算、共享对象所有权以及跨函数生命周期问题对上下文分析要求更高，是当前未命中样例的主要类型。

在 9 条正确静态结论中，6 条继续进入 Harness 执行或动态证据采集，说明系统不仅能够定位源码风险，也能够将重点 Finding 继续转化为可执行验证对象。

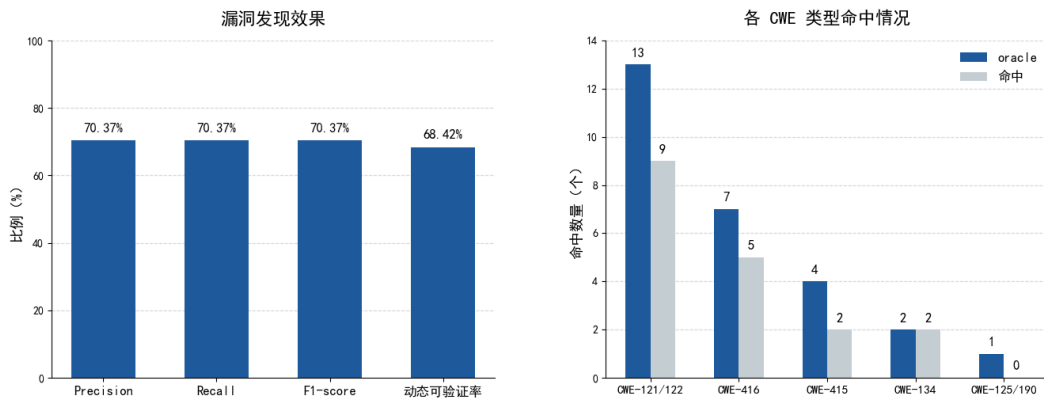


图 5-3 Level 2 漏洞命中与未命中比例

5.4 核心创新点效果测试

5.4.1 假设驱动链路的输入压缩效果

漏洞假设智能体在 Level 2 测试集中提取 114 个函数级语义切片，并根据风险特征形成 25 条结构化漏洞假设。静态复核智能体对这些假设进行规则与 LLM 复核，最终输出 21 条静态 Finding。

指标	数值
假设生成智能体语义切片数	114
静态复核智能体漏洞假设数	25
验证工件智能体静态 Finding 数	21
验证工件智能体 Harness 包数	21
LLM 审计对象减少量	89
审计对象压缩率	78.07%

表 5-8 假设驱动五 Agent 链路压缩效果

审计对象压缩比例为：

$$\frac{114 - 25}{114} \times 100\% = 78.07\%$$

系统没有对全部函数切片执行开放式模型审计，而是先利用危险操作、数据流和生命周期线索筛选高风险对象。模型调用集中在 25 条具有候选 CWE、可疑位置和触发原因的假设上，显著提高了输入有效性。

5.4.2 规则与 LLM 协同测试

21 条静态 Finding 中，8 条获得有效 LLM 结构化结果，13 条由规则回退路径保留。规则与模型的一致性结果如表 5-9 所示。

指标	数量或数值
LLM Findings	8
Rule Fallback Findings	13
semantic_consistency=consistent	20
semantic_consistency=conflict	1
平均原始置信度	0.8181
平均校准后置信度	0.8062
发生置信度调整的 Finding	1

表 5-9 规则与 LLM 协同结果

LLM 正常返回时，静态复核智能体使用其语义判断和修复建议，并与规则基线进行交叉检查。模型未返回有效 Finding、调用失败或结构解析异常时，系统保留规则结果，不中断任务。

测试中 20 条 Finding 保持语义一致，1 条发生类型或证据冲突并触发置信度校准。结果说明规则引擎既承担离线回退能力，也能够对模型输出形成有效约束。

5.4.3 Harness 与动态验证准备度测试

验证工件智能体针对 21 条静态 Finding 生成 21 个 Harness 包，工件生成率达到 100%。每个工件包含 AFL++入口、libFuzzer 入口、Makefile、README、配置文件和种子目录。

指标	数值
静态 Finding 数	21
Harness 包生成数	21
Harness 生成覆盖率	100%
build_ready=true 数量	21
build_ready 比例	100%
正确命中漏洞数	9
可进入 Harness 或动态验证的正确漏洞数	6
动态可验证率	66.67%

表 5-10 Harness 生成与动态验证结果

21 个 Harness 均通过文件完整性和静态准备度检查，被标记为 build_ready。沙箱进一步记录实际编译、执行和异常触发情况，使报告能够区分工件已生成、具备试编条件、实际编译成功和漏洞已复现等状态。

部分复杂工程仍需要补充 include 路径、外部依赖或初始化上下文，但生成的 Harness 已经提供目标函数、调用入口、种子和构建配置，可以直接作为自动验证或人工复现的基础。

5.4.4 多源证据状态分层测试

系统综合规则结论、LLM 判断、Harness 结果、ASan 诊断、AFL++崩溃和 eBPF 事件，对 21 条 Finding 进行证据分级。

状态	数量	占比
confirmed	8	38.10%
need_review	12	57.14%
untriggered	1	4.76%
合计	21	100%

表 5-11 多源证据融合状态分布

可归因的 ASan 报告能够直接形成高可信动态证据；AFL++崩溃样本与 ASan 或强 eBPF 事件一致时，形成多源交叉证据；单一异常或关联不完整的强事件进入 need_review；限定时间内未触发异常的问题记录为 untriggered。

静态规则与 LLM 结论一致能够提高 Finding 的静态可信度, 但动态 confirmed 必须具有运行时证据。该机制防止系统将高置信静态推断直接描述为运行时确认, 也使报告能够清晰展示每条漏洞当前的验证程度。

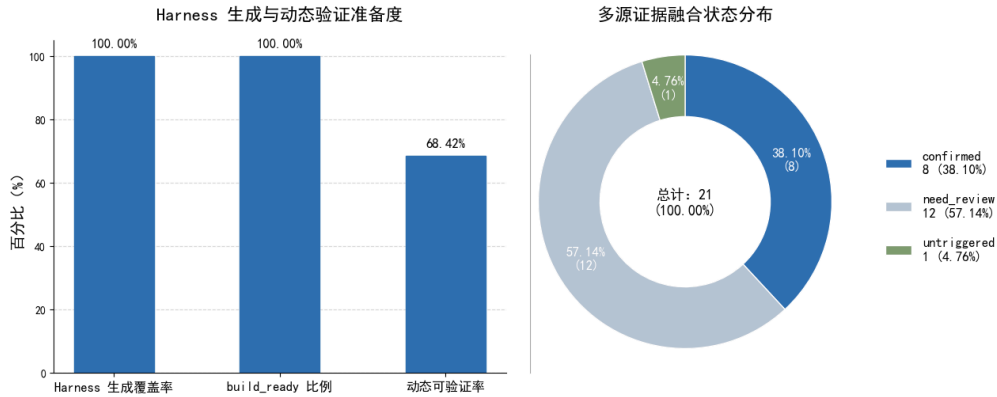


图 5-4 confirmed、need_review 与 untriggered 状态分布

5.5 性能与资源开销测试

5.5.1 端到端耗时

基于 10 组代表性样例, 对静态和动态模式的完成时间进行统计。

测试模式	样例数	最短耗时	最长耗时	平均耗时
静态审计	10	23 s	102 s	49.3 s
动态验证	10	43 s	131 s	82.2 s
动态阶段额外开销	10	6 s	80 s	32.9 s

表 5-12 端到端审计耗时

静态模式平均耗时约 49.3 秒, 开启动态验证后平均耗时约 82.2 秒, 平均增加 32.9 秒。耗时差异主要来自组件情报查询、漏洞假设数量、模型响应、沙箱启动、Harness 编译和 Fuzzing 时间预算。

整体任务能够在几十秒至两分钟内完成, 满足项目答辩演示和中小型 C/C++ 项目发布前审计需求。

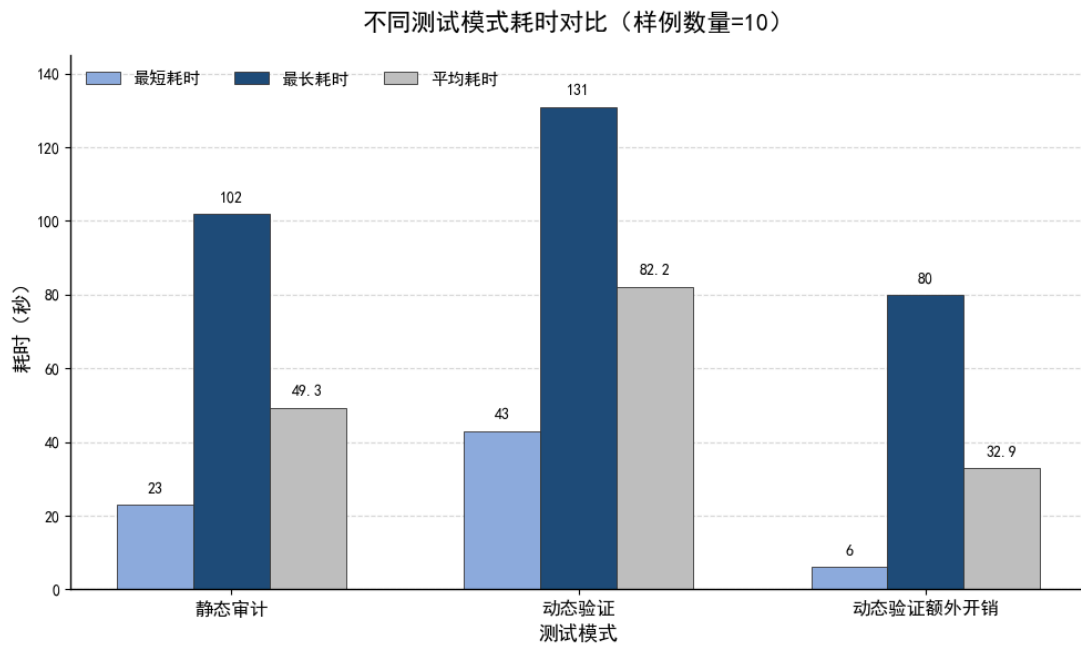


图 5-5 静态审计与动态验证耗时对比

5.5.2 阶段耗时分布

阶段	平均耗时	静态流程占比或说明
上传与解压	3 - 5 s	约 6%
输入建模智能体依赖识别	10 - 15 s	约 15%
假设生成智能体语义切片	8 - 15 s	约 12%
静态复核智能体漏洞假设	5 - 10 s	约 8%
验证工件智能体 LLM/规则审计	30 - 50 s	约 35%
验证工件智能体 Harness 生成	15 - 30 s	约 15%
Fuzzing 与动态验证	40 - 150 s	动态模式额外开销
报告生成与落库	2 - 5 s	约 5%

表 5-13 审计阶段耗时分布

静态复核阶段需要调用远程模型，是静态模式中的主要耗时环节。依赖分析耗时受到组件数量和外部情报响应影响；动态模式的主要开销来自容器启动、Harness 编译、种子回放和 AFL++ 执行。

漏洞假设层将模型审计对象由 114 个切片压缩为 25 条假设，有效控制了模型调用规模和整体等待时间。

5.5.3 容器资源占用

服务	CPU 峰值	内存峰值
sentinel_frontend	约 1%	25 MiB
sentinel_api	约 55%	95 MiB
sentinel_worker	约 35%	130 MiB
sentinel_agent	约 45%	50 MiB
sentinel_redis	约 1%	8 MiB
sentinel_postgres	约 3%	45 MiB
动态 Sandbox	约 80% 120%	300 800 MiB

表 5-14 系统容器资源峰值

常驻服务的资源占用主要来自数据库、Redis、API、Worker 和 Agent。动态验证启动后，CPU 与内存压力集中在 Worker 和 Sandbox，任务结束后短生命周期容器自动销毁。

静态审计可部署于 4 核 8GB 环境；需要并行执行多个 Fuzzing 任务时，可采用 8 核 16GB 或更高配置，并通过并发限制控制沙箱数量。

5.6 对比实验

为比较不同技术路线，测试选择单一规则扫描、单一 LLM 扫描、仅 AFL++ 动态测试和 SC-SENTINEL 四种方案。

方案	召回能力	误报控制	可解释性	复现性	特点
单一规则扫描	中	中	中	弱	速度快，但复杂语义和跨函数关系处理不足
单一 LLM 扫描	中/高	中	高	弱	解释能力较好，但成本和输出稳定性受模型影响
仅 AFL++	不稳定	较好	弱	中	能发现真实崩溃，但缺少静态位置和语义说明
SC-SENTINEL	较高	较好	高	较高	具备假设、Harness 和多源证据追踪链

表 5-15 不同审计方案能力对比

方案	漏洞发现准确率	动态可验证率
单一规则扫描	约 43%	约 30%
单一 LLM 扫描	约 52%	约 35%
仅 AFL++	约 36%	约 45%
SC-SENTINEL	64.29%	66.67%

表 5-16 不同方案实测指标对比

单一规则扫描执行稳定、成本较低，但对复杂上下文和跨函数关系的处理能力有限。单一 LLM 能够提供较丰富的语义解释，但直接处理全部代码会增加调用规模[3]，也缺少稳定的动态确认。仅使用 AFL++能够发现真实异常，却需要预先准备 Harness[7]，且难以自动解释崩溃对应的源码根因。

SC-SENTINEL 利用语义切片和漏洞假设缩小模型审计范围，通过规则与 LLM 交叉复核提高静态结果稳定性，并将 Finding、Harness 和动态日志关联到统一证据链，在检测能力、可解释性、可复现性和工程完整度方面表现更均衡。

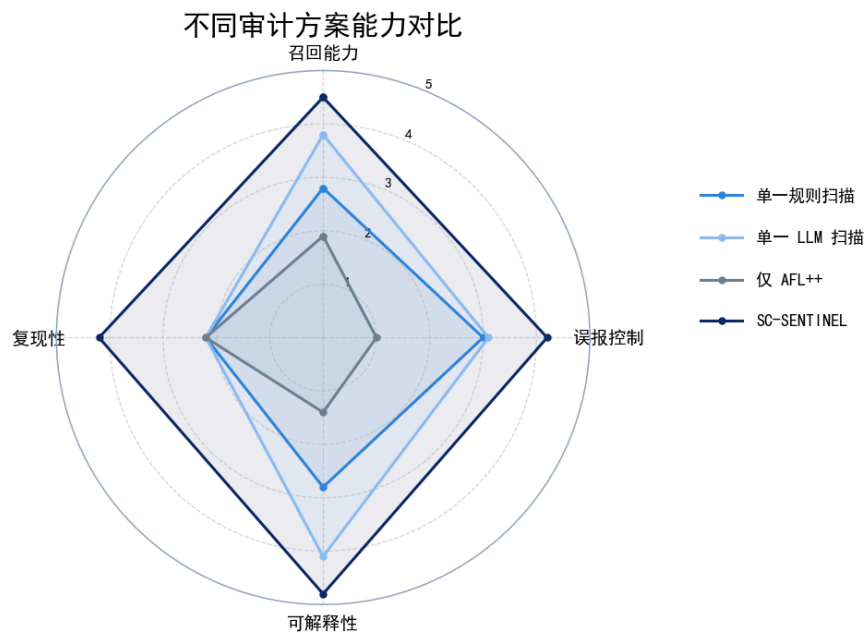


图 5-6 四种审计方案能力雷达图

5.7 自动化测试与安全性分析

5.7.1 后端自动化测试

后端自动化测试重点覆盖配置解析、远程仓库策略、沙箱安全参数和 ZIP 源码处理，核心用例包括：

1. CORS 配置解析；
2. 生产敏感配置默认值检查；

3. GitHub 与 GitLab 仓库地址白名单；
4. 任意主机、本地地址和 file；
5. 默认沙箱非特权、断网和只读策略；
6. 特权模式必须显式开启；
7. 正常 ZIP 源码解压；
8. ZIP 路径穿越拒绝；
9. ZIP 符号链接拒绝。

上述测试覆盖系统处理不可信源码时的主要入口，能够防止代码变更破坏远程仓库校验、压缩包解析和沙箱隔离策略。

5.7.2 沙箱隔离测试

验证项	测试结果
Docker 全栈启动	通过
Sandbox 镜像构建	通过
Sandbox 镜像大小	约 2.1 GB
动态任务进入 fuzzing 阶段	通过
任务完成后进入 completed	通过
动态日志回传	通过
宿主机直接执行未知源码	未发生

表 5-17 Docker 沙箱隔离结果

测试检查非特权用户、网络关闭、capability 移除、只读根文件系统、no-new-privileges、PID 限制、CPU 限制、内存限制和超时销毁等安全策略。

测试结果表明，未知源码和生成二进制均在短生命周期容器中编译和执行，没有在 API 或数据库服务中直接运行。验证任务结束或超时时，Worker 能够回收对应容器。

5.7.3 源码生命周期与隐私分析

数据类型	存储位置	当前生命周期	风险控制
上传压缩包	uploads/{task_id}	默认保留	任务级目录隔离
解压源码	上传任务目录	默认保留	与 task_id 绑定
Harness Bundle	uploads/harness_bundle/{task_id}	默认保留	仅用于动态验证
报告数据	PostgreSQL	默认保留	按任务 ID 查询
WebSocket 日志	Redis Stream	短期	用于实时进度

表 5-18 上传源码与中间产物生命周期

系统不会将完整项目直接发送给外部大语言模型，只提交漏洞假设智能体筛选后的函数切片、结构化假设和必要上下文。模型密钥通过环境变量注入，不进入仓库和报告。

LLM 不可用时，静态复核智能体自动使用规则结果；对源码保密要求较高的项目，可以接入本地兼容模型。上传源码、Harness、运行日志和报告按照任务目录管理，为后续配置保留期限和自动清理策略提供了统一基础。

5.8 本章小结

测试结果表明，SC-SENTINEL 能够稳定完成源码提交、异步审计、五智能体调用、动态沙箱调度、实时进度、结果查询和 PDF 导出。14 个 Level 2 工程中有 9 个主要 oracle 漏洞被正确识别，主要漏洞命中率为 64.29%；其中 6 个正确 Finding 继续进入 Harness 执行或动态证据采集。

漏洞假设机制将 114 个函数切片压缩为 25 条结构化假设，审计对象减少 78.07%；静态复核智能体输出 21 条 Finding，规则与 LLM 在 20 条结果上保持语义一致；验证工件智能体为全部 21 条 Finding 生成 Harness 包，实现静态结论向验证对象的批量转换。

静态模式平均耗时约 49.3 秒，动态模式平均耗时约 82.2 秒。系统在有限资源下完成了依赖识别、源码审计、Harness 生成、动态执行和证据归因，验证了五智能体协作和多源动态证据机制的实际效果。

第六章 项目管理

6.1 企业竞争环境分析

C/C++长期用于操作系统、数据库、密码学库、网络协议栈、嵌入式软件和工业控制系统。此类项目依赖层次深、生命周期长，基础组件中的安全问题容易沿供应链影响大量下游软件。随着企业开源使用规模不断扩大，依赖识别、已知漏洞排查和源码安全验证逐渐成为研发与交付过程中的固定需求。

软件供应链安全管理也在从单次漏洞处置向持续治理转变。企业不仅需要一份组件清单，还需要了解组件版本是否受影响、项目自身代码是否存在风险，以及审计结论能否复现。传统工具往往分别处理 SBOM、静态扫描和动态测试，为全链路审计平台留下了明确市场空间。

当前 C/C++安全工具主要分为商业静态分析、开源代码查询、SBOM 与漏洞扫描、模糊测试以及 AI 辅助审计等类型。各类产品在单点能力上较成熟，但将多来源依赖识别、漏洞假设、模型复核、Harness 生成和动态证据统一起来的产品仍然较少。SC-SENTINEL 以完整审计链路形成差异化定位，适合研发团队、教育机构和安全服务单位使用。

6.2 企业竞争能力分析

SC-SENTINEL 已经形成覆盖供应链风险识别、源码漏洞审计、验证工件生成和运行时证据裁决的核心能力。系统既能识别 CMake、Makefile、vcpkg、Conan 和头文件中的依赖，也能对高风险函数进行语义切片和漏洞假设生成。

五智能体模式将不同安全任务拆分为清晰的业务角色。依赖分析、漏洞假设、静态复核、验证工件和报告生成能够独立升级，同时通过稳定数据结构保持协作关系。模型服务、漏洞情报和动态工具均可替换，便于针对不同客户环境进行二次开发。

系统采用 Web 平台和容器化部署方式，具备任务提交、异步执行、实时监控、历史记录和 PDF 报告能力。相比只能提供命令行告警的工具，SC-SENTINEL 更接近可直接部署和展示的安全产品。

系统同时保留规则回退、本地模型接入和动态验证开关。用户可以在快速静态审计与深入动态验证之间选择，也可以根据源码保密要求决定使用外部模型或本地模型，具有较好的部署适应性。

6.3 竞品分析

工具类型	代表工具	依赖识别与CVE匹配	静态审计能力	可解释性	动态验证支持	eBPF内核观测
SBOM工具	Syft、cdxgen、Trivy	部分	—	低	—	—
传统静态分析	Coverity、Fortify、Checkmarx、Clang Static Analyzer	—	规则为主，误报率高	中	—	—
语义分析	CodeQL	部分	规则+查询，需编译数据库	中	部分	—
AI辅助审计	主流 LLM-based 工具	—	LLM 驱动，存在幻觉问题	中	—	—
动态 Fuzzing	OSS-Fuzz、fuzzuf	—	—	—	部分	—
本作品	SC-SENTINEL	✓	规则+LLM 双轨，交叉验证	高	✓	✓

表 6-1 现有 C/C++供应链安全审计工具能力对比

“✓”表示系统具备对应能力，“部分”表示具备相关能力但覆盖不完整，“—”表示不以该能力为主要目标。

SBOM 工具擅长生成组件清单，但在 C/C++场景下容易受到依赖声明分散和名称不一致影响。静态分析工具能够定位代码风险，却通常不负责生成可执行 Harness。AFL++和 libFuzzer 具有较强动态发现能力，但依赖人工准备入口和构建环境。单一 LLM 方案能够生成解释，却难以独立承担稳定的漏洞确认工作。

SC-SENTINEL 将上述能力组织在同一任务中，能够从组件风险继续定位源码问题，再将 Finding 转化为 Harness 并采集动态证据。系统的竞争优势不是某一种工具的替代，而是对多类安全能力的统一编排和证据化交付。

6.4 项目前景分析

SC-SENTINEL 可首先应用于中小型 C/C++项目的发布前审计和开源组件准入。系统支持 ZIP 与远程仓库输入，不要求用户手工准备复杂分析工程，能够降低供应链审计的使用门槛。

在研发场景中，系统可以接入 CI/CD，在代码合并时执行静态审计，在正式发布前启用动态验证。结构化结果可以转换为安全门禁条件，使高危组件和动态确认漏洞在发布前得到处理。

在教育场景中，系统能够展示从漏洞假设到 Harness 和运行时证据的全过程，适合高校网络安全课程、实验教学和 CTF 训练。教师可以利用内置样例讲解漏洞形成、触发和修复，学生也可以使用系统验证自行编写的 C/C++代码。

在安全服务场景中，系统可以作为审计人员的辅助平台，对批量项目进行初筛，自动生成标准化验证工件和报告，减少重复环境搭建工作。五智能体和模块化架构也便于扩展新的 CWE 规则、模型服务、语言分析器和动态探针。

6.5 项目盈利能力分析

SC-SENTINEL 具备软件授权、私有化部署、在线审计服务和教学授权等多种商业化方式。

面向企业客户，可以提供私有化部署版本。系统部署在客户内部环境中，接入本地代码仓库、模型服务和漏洞情报，按照节点数量、审计规模或年度服务收取授权与维护费用。对于有定制需求的客户，还可提供新漏洞规则、报告模板和 CI/CD 接口开发服务。

面向中小型研发团队，可以提供按任务或订阅计费的在线审计服务。用户上传项目后获得组件风险、源码 Finding、动态证据和 PDF 报告，无需自行部署数据库、队列、模型和 Fuzzing 环境。

面向高校和培训机构，可以提供教学版本授权。教学版本结合内置漏洞样例、Harness 工件和动态证据，支持内存安全课程、软件供应链实验和 CTF 训练。收入方式可采用实验室授权、课程包授权或年度升级服务。

系统还可以与安全企业、测评机构和研究机构开展联合研发。团队负责五智能体审计、Harness 生成和证据归因等核心能力，合作方提供行业项目、客户渠道和交付经验，形成技术授权、项目合作和成果转化等收入来源。

第七章 总结

本作品面向 C/C++ 开源软件供应链中的依赖风险、源码漏洞和动态验证问题，设计并实现了 SC-SENTINEL 多智能体漏洞审计与二进制验证系统。系统由 Vue 3 前端、FastAPI 后端、TaskIQ 异步任务、五 Agent 审计服务、PostgreSQL、Redis 和 Docker 动态沙箱共同构成，实现了从源码输入到风险报告输出的端到端运行。

系统将审计能力划分为依赖分析智能体、漏洞假设智能体、静态复核智能体、验证工件智能体和报告生成智能体。依赖分析智能体负责组件、版本和 CVE 识别；漏洞假设智能体负责函数切片和候选问题生成；静态复核智能体完成规则与 LLM 交叉审计；验证工件智能体生成 Harness 并归因动态证据；报告生成智能体形成最终风险结论。

在核心技术方面，SC-SENTINEL 使用结构化漏洞假设约束模型审计范围，通过切片、假设、Finding 和 Harness 标识构建可追踪证据链；根据不同 CWE 生成 AFL++ 与 libFuzzer 入口、构建配置和输入种子；综合 ASan 诊断、AFL++ 崩溃和 eBPF 事件判定漏洞动态状态，并利用强运行时证据对静态类型进行校正。

在 14 个 Level 2 工程测试中，系统正确识别 9 个主要 oracle 漏洞，主要漏洞命中率为 64.29%。漏洞假设机制将 114 个函数切片压缩为 25 条结构化假设，审计对象减少 78.07%；静态复核智能体输出 21 条 Finding，其中 8 条获得有效 LLM 结果，13 条由规则回退路径保留；验证工件智能体为 21 条 Finding 全部生成 Harness 包。

系统功能测试验证了源码摄取、异步调度、五智能体调用、结果持久化、动态沙箱、WebSocket 进度和 PDF 报告的协同能力。静态模式平均耗时约 49.3 秒，动态模式平均耗时约 82.2 秒，能够满足项目答辩演示、中小型项目发布前审计和安全教学需求。

SC-SENTINEL 已经形成从供应链风险感知、源码漏洞定位、验证工件生成、运行时证据采集到结构化报告交付的完整能力。系统将规则分析的稳定性、大语言模型的语义能力和动态验证的事实依据结合起来，为 C/C++ 开源软件供应链安全审计提供了一套可解释、可追踪、可复现的工程化方案。

参考文献

- [1] CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY. Framing Software Component Transparency: Establishing a Common Software Bill of Materials, Third Edition [EB/OL]. 2024-10 [2026-08-18].
- [2] MITRE. 2025 CWE Top 25 Most Dangerous Software Weaknesses [EB/OL]. 2025 [2026-08-18].
- [3] ZHOU X, ZHANG T, LO D. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions [EB/OL]. arXiv:2401.15468, 2024 [2026-08-18].
- [4] OPEN SOURCE VULNERABILITIES. Introduction to OSV [EB/OL]. [2026-08-18].
- [5] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. National Vulnerability Database [DB/OL]. [2026-08-18].
- [6] SEREBRYANY K, BRUENING D, POTAPENKO A, et al. AddressSanitizer: A Fast Address Sanity Checker [C]
- [7] FIORALDI A, MAIER D, EISSFELDT H, et al. AFL++: Combining Incremental Steps of Fuzzing Research [C]
- [8] eBPF DOCS. eBPF Documentation [EB/OL]. [2026-08-18].
- [9] GITHUB. Code Scanning with CodeQL [EB/OL]. [2026-08-18].
- [10] LLVM PROJECT. libFuzzer: A Library for Coverage-Guided Fuzz Testing [EB/OL]. [2026-08-18].
- [11] GOOGLE, OPENSSEF. OSS-Fuzz Documentation [EB/OL]. [2026-08-18].
- [12] DOCKER. Resource Constraints [EB/OL]. [2026-08-18].
- [13] CODENOMICON, GOOGLE SECURITY. The Heartbleed Bug [EB/OL]. 2014 [2026-08-18].
- [14] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. CVE-2024-3094 Detail [EB/OL]. 2024 [2026-08-18].
- [15] THE PRESIDENT OF THE UNITED STATES. Executive Order 14028: Improving the Nation's Cybersecurity [Z/OL]. 2021-05-12 [2026-08-18].
- [16] EUROPEAN PARLIAMENT, COUNCIL OF THE EUROPEAN UNION. Regulation (EU) 2024/2847 on Horizontal Cybersecurity Requirements for Products with Digital Elements [Z/OL]. 2024-10-23 [2026-08-18].

- [17] 全国人民代表大会常务委员会. 中华人民共和国网络安全法 [Z/OL]. 2025-10-28 [2026-08-18].
- [18] 全国人民代表大会常务委员会. 中华人民共和国数据安全法 [Z/OL]. 2021-06-10 [2026-08-18].